



Objektmodell

WinLine Edition 2024 - Version 12.24

Inhaltsverzeichnis

1.	Objekthierarchie	4
2.	Objekte	5
3.	MacroCommands	7
4.	Verwendung von Modulen	8
5.	Beschreibung der Objektmodelle	10
5.1.	CWLStart	10
5.1.1.	Eigenschaften	11
5.1.2.	Methoden	13
5.1.3.	Events	15
5.2.	CWLScript	18
5.2.1.	Eigenschaften	18
5.2.2.	Methoden	19
5.2.3.	Events	19
5.3.	CWLCurrentModule	19
5.3.1.	Eigenschaften	20
5.3.2.	Events	20
5.4.	CWLCurrentWindow	20
5.4.1.	Eigenschaften	21
5.4.2.	Events	21
5.5.	CWLWindowVars	25
5.5.1.	Eigenschaften	25
5.5.3.	Methoden	27
5.6.	CWLEventResult	27
5.6.1.	Eigenschaften	27
5.7.	CWLSearchResult	28
5.7.1.	Eigenschaften	28
5.7.2.	Methoden	29
5.8.	GeneralScriptFuncs	30
5.8.1.	Eigenschaften	30
5.8.2.	Methoden	30
5.9.	CWLTable	33
5.9.1.	Eigenschaften	33
5.9.2.	Methoden	33
6.	Klassen	37
6.1.	CWLCompany	37
6.1.1.	Eigenschaften	37
6.1.2.	Methoden	39
6.1.3.	Events	41
6.2.	CWLDbConnection	42
6.2.1.	Eigenschaften	42
6.2.2.	Methoden	43
6.3.	CWLModule	45
6.3.1.	Eigenschaften	46
6.3.2.	Methoden	46
6.3.3.	Verwendung	48
6.4.	CWLWinCollection	48
6.4.1.	Eigenschaften	48
6.4.2.	Methoden	48
6.4.3.	Verwendung	49
6.5.	CwlWindow	50
6.5.1.	Eigenschaften	50
6.5.2.	Methoden	51

6.6.	CwlFgCollection	53
6.6.1.	Eigenschaften	53
6.6.2.	Methoden	53
6.6.3.	Verwendung	54
6.7.	CwlFgControl	54
6.7.1.	Eigenschaften	55
6.7.2.	Methoden	57
6.8.	CwlPreview	60
6.8.1.	Eigenschaften	61
6.8.2.	Methoden	61
6.9.	CwlPreviewPage.....	62
6.9.1.	Eigenschaften	62
6.9.2.	Methoden	62
6.10.	CwlPreviewPageItem.....	62
6.10.1.	Eigenschaften	63
6.11.	CwlSpreadSheet.....	64
6.11.1.	Eigenschaften	64
6.11.2.	Methoden	64
6.12.	CWLGrid	66
6.12.1.	Eigenschaften	67
6.12.2.	Methoden	68
6.13.	CWLReport	81
6.13.1.	Eigenschaften	82
6.13.2.	Methoden	84
6.13.3.	Events.....	85
7.	Konstanten	87
7.1.	CWLApplicationNr	87
7.2.	CWLWindowTypes	87
7.3.	CWLControlTypes.....	87
7.4.	CWLSpoolItemType.....	88
7.5.	CWLSpoolPreviewItemFlag	89
7.6.	CWLAlignements.....	89
7.7.	CWLScriptWindowType	89
7.8.	CWLSystemServerType	90
7.9.	CWLDbConnectionType	90
7.10.	CWLGridColumnFlags	90
8.	Tipps und Tricks.....	92
8.1.	Bearbeiten von Scripts bei Kundeninstallationen.....	92
8.2.	Kann von extern ein WinLine-Fenster angesprochen werden?	92
8.3.	Makros aus einem Script heraus aufrufen	92
8.4.	Optionale Steuerung der rechten Maustaste	93

1. Objekthierarchie

CWLStart		
		CurrentCompany (CWLCompany)
		CWLCurrentcompany
		Module (CWLM Modul)
		CurrentWindow (CWLWindow)
		CWLModul
		CurrentWindow (CWLWindow)
		Windows (CWLWindowCollection)
		Item (CWLWindow)
		NamedItem (CWLWindow)
		IndexedItem (CWLWindow)
		CWLWindow
		Vars (CWLWindowVars)
		CWLWindowVars
		Controls (CWLFGCollection)
		Item (CWLFGControl)
		IndexedItem (CWLFGControl)
		CurrentControl (CWLFGControl)
		CWLFGControl
		Preview (CWLPreview)
		CWLPreview
		Page (CWLPreviewPage)
		CWLPreviewPage
		Item (CWLPreviewPageItem)
		CWLPreviewPageItem
		SpreadSheet (CWLSpreadSheet)
		CWLSpreadSheet
		Bildschirmtabelle (CWLGrid)
		CWLGrid
		CWLReport
CWLCurrentModule		
		ActiveModule (CWLModule)
CWLCurrentWindow		
		ActiveWindow (CWLWindow)
CWLScript		
CWLMacro		
CWLEventResult		
GeneralScriptFuncs		
LOHNFormel		
FAKTFormel		

2. Objekte

Folgende eingebaute Objekte können in CWL VBScript verwendet werden:

Objektname	Verwendung
CWLCurrentModule	Steht nur in CTK-Scripts zur Verfügung. Die einzige Property ActiveModule entspricht CWLStart.CurrentModule . Das Objekt stellt die Eventschnittstelle für die modulspezifischen Events dar. Es entspricht zu jedem Zeitpunkt dem gerade aktiven Modul.
CWLCurrentWindow	Steht nur in CTK-Scripts zur Verfügung. Die einzige Property ActiveWindow entspricht CWLStart.CurrentWindow . Das Objekt stellt die Eventschnittstelle für die fensterspezifischen Events dar. Es entspricht zu jedem Zeitpunkt dem gerade aktiven Fenster.
CWLScript	Repräsentiert das Script. Steht in Lohn-Scripts, CTK-Scripts und System-Scripts zur Verfügung.
CWLStart	Dient zur Steuerung der Applikation. Steht nur in Systemscripts, CTK-Scripts, CRM-Scripts und FAKT-Formeln zur Verfügung.
FormDriver	Wird nur intern verwendet.
MacroCommands bzw. CWLMacro	Stellt alle wichtigen Funktionen für die Makroverarbeitung zur Verfügung. Steht in allen Scripts zur Verfügung
UserForm	Repräsentiert das UserForm. Steht nur in Scripts zur Verfügung, die ein UserForm verwenden: CTK-Scripts, System-Scripts, Lohn-Scripts Kann dazu verwendet werden um direkte Aktionen mit dem Skriptfenster durchzuführen (z.B. Reaktion auf Mausklicks in das Fenster)
CWLWindowVars	Zugriff auf Fenstervariable
CWLEventResult	Rückgabe von Ergebnissen in Events, die dies Unterstützen
CWLSearchResult	Enthält das Ergebnis einer SQL Abfrage (siehe CWLCompany - Objekt)
GeneralScriptFuncs	<i>MsgBox</i> und <i>InputBox</i> auch für die Scripts in der EWL (die VBScript Funktionen werden in der EWL am Server ausgeführt, sodass der Client nichts davon mitbekommt). Mit <i>FileDialog</i> können Dateieingaben ausgeführt werden und mit <i>WaitCursor</i> die Sanduhr angezeigt werden. Mit der Funktion <i>Convert</i> kann Text nach Base64, zurück von Base64 und RTF-Text nach PlainText konvertiert werden. Mit <i>MsgWin</i> , <i>MsgWinSetText</i> und <i>MsgWinDestroy</i> kann während einer länger dauernden Schleife ein kleines Fenster mit einem Text dargestellt werden, optional mit einem Abbruchbutton.

CWLTable	Repräsentiert eine geöffnete Tabelle. Wird von einem CWLDbConnection Objekt erzeugt.
CWLReport	Mit diesem Objekt kann eine Auswertung in der CWL selbst programmiert werden. Das Objekt kann von einem CWLWindow erzeugt werden.

3. MacroCommands

Dieses Objekt wird hauptsächlich in der Makroverarbeitung verwendet, da es auch im Makrorecorder verfügbar ist. Einige der Funktionen die über dieses Objekt verfügbar sind, können auch mit anderen CWL Objekten realisiert werden.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros
- ☐ Makro Recorder
- ☐ LOHN Makros
- ☐ FAKT Makros

4. Verwendung von Modulen

Mit dem Schlüsselwort **\$IMPORT:scriptname,scriptname,....** kann in ein Script ein anderes Script (nur Systemscrip te) importiert werden, um Funktionen, die in mehreren Scripten verwendet werden sollen, nur einmal erfassen zu müssen.

Das Schlüsselwort kann nur innerhalb eines Remarks angegeben werden, da es keine offizielle VBScript - Funktion ist.

Alle importierten Scripts müssen innerhalb einer Zeile angegeben werden, abgeschlossen durch einen Zeilenumbruch.

Die angeführten Scripts werden bei Ausführung des Scripts automatisch nach der Zeile mit dem Schlüsselwort eingefügt.

Events, die in den eingefügten Scripts vorhanden sind, werden nicht ausgeführt und sollten in Library-Scripts nicht verwendet werden.

Nur in Fenster- und Systemscrip ts kann das \$IMPORT verwendet werden, in Makros wird es ignoriert.

Beispiel:

Es gibt zwei Systemscrip ts, die als Module in einem Testscript eingefügt werden.

Der Inhalt des 1. Moduls (LIB1):

```
' (Deklarationen)
    Function lib1
        lib1 = "Das ist Lib1"
    End Function

'Ende von (Deklarationen)
```

Der Inhalt des 2. Modules (LIB2):

```
' (Deklarationen)
    Function lib2
        lib2 = "das ist lib2"
    End Function

'Ende von (Deklarationen)
```

Das Script, das die beiden Module verwendet:

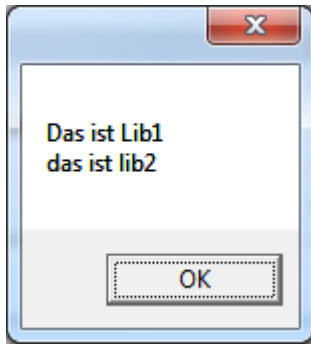
```
'(Deklarationen)

' $IMPORT:LIB1,LIB2

'Ende von (Deklarationen)

Sub CWLScript_OnScriptStart()
    msgbox lib1 () & chr(13)&chr(10) & lib2 ()
End Sub
```

Der Aufruf des Testscrip ts würde dann die folgende Ausgabe ergeben:



5. Beschreibung der Objektmodelle

5.1. CWLStart

Dieses Objekt dient zur Steuerung der gesamten Applikation.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros
- ☐ CRM-Scripts
- ☐ FAKT-Formeln

Achtung: Das Objekt ist das Defaultobjekt in diesen Scripts und alle Properties und Methoden können auch ohne das Prefix **CWLStart.** angesprochen werden.

Name ist somit beispielsweise keine benutzerdefinierte Variable sondern entspricht **CWLStart.Name** (es sei denn, man definiert **Name** zuvor mit **Dim**).

CWLStart

Eigenschaften

ICwlStart* **Application**
 BSTR **FullName**
 BSTR **Name**
 BOOL **Visible**
 ICWLModule* **CurrentModule**
 ICwlWindow* **CurrentWindow**
 BSTR **AppPath**
 BSTR **WorkPath**
 BSTR **ServerPath**
 ICWLCompany * **CurrentCompany**
 VARIANT **GlobalProperty** (int PropertyNr)
 ICWLDbConnection ***Connection**
 ICWLUser ***CurrentUser**
 ICWLInvoicingModule ***InvoicingModule**
 ICWLMacro* **MacroCommands**
 short **SessionType**
 BSTR **WebserviceResult**

Methoden

ICWLModule* **Module** (short nApplicationNr)
 void **Quit** ()
 BOOL **ActivateModule** (CWLApplicationNr nApplicationNr)
 BOOL **ActivateExternalApp** (short nApplicationId)
 BOOL **ExecuteMacro** (BSTR strMacroName)
 BOOL **RunFormScript** (BSTR strScriptName, CWLScriptWindowType mode)
 BOOL **SendMail** (BSTR addr, BSTR strSubject, BSTR strText, BSTR attachments, BOOL bWithDialog)
 void **SetAppBackgroundPic** (BSTR picture, int mode)
 void **SetAppBackgroundColor** (short red, short green, short blue)
 void **SetDefaultWinColor** (short red, short green, short blue)

Events

OnQuit (ICwlEventResult *bResult)
OnActivateApp (CWLApplicationNr AppNr)

OnWindowOpen (CWLApplicationNr AppNr, int windowId)
 OnWindowClose (CWLApplicationNr AppNr, int windowId)
 OnWindowActivate (CWLApplicationNr AppNr, int windowed)
 OnScriptWindowMayClose (CWLApplicationNr AppNr, int windowId, ICwlEventResult *bResult)
 OnPagePrinted (CWLApplicationNr AppNr, int windowId, int controlWinId, int controlId, int PageNr, BSTR formName)
 OnPageStarted (CWLApplicationNr AppNr, int windowId, int controlWinId, int controlId, int PageNr, BSTR formName)
 OnMessageBoxLaunched (CWLApplicationNr AppNr, int windowId, BSTR MessageBoxText, ICwlEventResult *ButtonPressed)
 OnBeforeMessageBoxLaunched (CWLApplicationNr AppNr, int windowId, BSTR MessageBoxText, ICwlEventResult *ButtonPressed)
 OnCompanyChange (BSTR CompanyNumber, int CompanyYear)

OnContextMenu (int, int, int, BSTR, int, ICwlEventResult *IsAllowed)

5.1.1. Eigenschaften

FullName [BSTR, read only]

Name der EXE-Datei mit Pfad, ohne Erweiterung.
z.B. "C:\WINLINE\cwlstart"

Name [BSTR, read only]

Name der EXE-Datei ohne Pfad, ohne Erweiterung.
z.B. " cwlstart"

AppPath [BSTR, read only]

Pfad der EXE-Datei, Backslash terminiert.
z.B. "C:\WINLINE\"

WorkPath [BSTR, read only]

Aktuelles Arbeitsverzeichnis der Applikation, Backslash terminiert.
z.B. "C:\WINLINE\"

ServerPath [BSTR, read only]

Pfad zum Serververzeichnis, Backslash terminiert. Dabei kann es sich auch um einen UNC Pfad handeln.
z.B. "C:\WINLINE\" oder "\\SERVER\WINLINE\"

Visible [BOOL, read write]

Regelt, ob das gesamte Fenster der CWL sichtbar ist oder nicht.

TRUE	Das Fenster ist unsichtbar.
FALSE	Das Fenster ist sichtbar

CurrentModule [ICWLModule*, read only]

Zeiger auf das aktuelle Modul (siehe Klasse ICWLModule).
Es gibt immer ein aktuelles Modul.

CurrentWindow [ICWLWindow*, read only]

Zeiger auf das aktuelle Fenster (siehe Klasse CWLWindow).

Es gibt immer ein aktuelles Fenster, zumindest das zum Script gehörende Userform (das unter Umständen unsichtbar ist).

Application [ICWLStart*, read only]

Die Applikation selbst.

GlobalProperty (int PropertyNr) [VARIANT, read write]

Setzt oder retourniert eine benutzerspezifische Eigenschaft, die mit einer beliebigen Nummer vom Benutzer gekennzeichnet ist. Die Eigenschaft dient zur Informationsübermittlung zwischen verschiedenen Skripts.

Das eine Skript setzt eine Property:

GlobalProperty (1) = "Ein Test"

Ein anderes Skript (oder auch das gleiche) kann die Property wieder auslesen:

[MsgBox](#) GlobalProperty (1)

In der GlobalProperty kann auch ein Objekt, das mit CreateObject () erzeugt wurde, gespeichert werden.

CurrentCompany [ICWLCompany*, read only]

Zeiger auf den aktuellen Mandanten (siehe Klasse CWLCompany).

Connection (VARIANT what) [ICWLDbConnection*, read only]

Erzeugt ein CWLDbConnection-Objekt abhängig vom übergebenen Parameter. Ist der Parameter ein String mit 4 Buchstaben, dann wird angenommen, dass dies eine Mandantennummer ist und es wird versucht die Verbindungsparameter des Mandanten zu lesen. Ist der Wert ein numerischer Wert zwischen 0 und 9 werden die Verbindungsparameter entsprechend des Parameters bestimmt:

- 0 der aktuelle Mandant
- 1 Systemdatenbank für Datenbankverbindungen, Benutzer, usw.
- 2 Systemdatenbank für Formulare und Fenster
- 3 Systemdatenbank der mandantenunabhängigen Daten
- 4 Systemdatenbank für Archivtabellen
- 5 Systemdatenbank für Lohndaten Österreich
- 6 Systemdatenbank für Lohndaten Deutschland
- 7 (nicht verwendet)
- 8 Systemdatenbank für Datenquellen (PowerReport)
- 9 Systemdatenbank für das Variablenaudit (nur, wenn dieses aus dem Mandanten ausgelagert wurde)

CurrentUser [ICWLUser*, read only]

InvoicingModule [ICWLInvoicingModule*, read only]

MacroCommands [ICWLMacro*, read only]

SessionType [int, read only]

Diese Eigenschaft liefert die folgende Werte:

in der CWL: 0

in der EWL: 1
in der MWL: 2
WebService: 3

WebServiceResult [BSTR, read/write]

Diese Eigenschaft kann nur in Scripts verwendet werden, die am Mesonic Server ausgeführt werden. Damit kann in Makros, die von WebServices ausgeführt werden, ein spezifisches Ergebnis festgelegt werden, das der Client nach dem Aufruf des Webservice - Makros als Ergebniswert erhält.

5.1.2. Methoden

Module(short nApplicationNr)

Parameter

NApplicationNr	Nummer des gewünschten Moduls (siehe CWLApplicationNr - Konstanten)
----------------	---

Rückgabewert (ICWLModule *)

Liefert einen Pointer auf das Modul mit der ID **nApplicationNr** (siehe auch Konstanten - CWLApplicationNr) zurück vom Typ **ICWLModule**.

Die jeweilige Applikation muß dazu nicht aktiv sein.

Quit

Beendet die Applikation.

Nicht gespeicherte Daten werden nicht gesichert!

ActivateModule(short nApplication)

Schaltet auf die entsprechende Applikation mit der ID **nApplication** (siehe dazu auch Konstanten unter **CWLApplicationNr**) um.

Siehe auch MacroCommands.MApplication.

Parameter

nApplication	Nummer des gewünschten Moduls (siehe CWLApplicationNr - Konstanten)
--------------	---

Rückgabewert (VARIANT_BOOL)

TRUE	Applikation konnte umgeschaltet werden
FALSE	Ungültiger Wert für nApplication , oder auf die Applikation konnte nicht umgeschaltet werden, weil z.B. der Benutzer keine Berechtigung hat

ActivateExternalApp(short nApplicationId)

Startet die als "Externes Programm" eingetragene Application mit der Nummer **nId**. Die ID der eingetragenen Programme wird mit 0 beginnend nummeriert.

Siehe auch MacroCommands.MexternalApplication

Parameter

nApplicationId	Index des gewünschten Programmes (0 bis 9)
----------------	--

Rückgabewert (VARIANT_BOOL)

TRUE	Applikation konnte gestartet werden
FALSE	Applikation konnte nicht gestartet werden

ExecuteMacro(BSTR strMacroName)

Ruft in **strMacroName** angegebenen Makro auf. Wird dieser Befehl in einem Skript verwendet, um ein anderes aufzurufen, dann wird zuerst das aufgerufene Skript ausgeführt und danach wieder das aufrufende Skript fortgesetzt.

Mit diesem Befehl können nur Makros, jedoch keine anderen Scripts aufgerufen werden!

Siehe auch MacroCommands.MRunMacro

Parameter

strMacroName	Der Name des Makros, das gestartet werden soll
--------------	--

Rückgabewert (VARIANT_BOOL)

TRUE	Makro konnte gestartet werden
FALSE	Makro konnte nicht gestartet werden

RunFormScript(BSTR strScriptName , CWLScriptWindowType mode)

Ruft in **strScriptName** angegebenes Systemskript auf. Wird dieser Befehl in einem Skript verwendet, um ein anderes aufzurufen, dann wird das aufgerufene Systemskript gestartet und bleibt aktiv. Das aufrufende Script bekommt die Kontrolle nach dem **FormDriver_OnActivate** Event des aufgerufenen Systemskripts zurück (siehe auch Events von SystemScripts).

Der Parameter **mode** beschreibt wie das Skript gestartet wird :

0 ... als Standardfenster das beim Modulwechsel versteckt wird

1 ... als modales Fenster

2 ... als Fenster das über allen anderen Fenster schwebt und beim Modulwechsel sichtbar bleibt)

Siehe auch MacroCommands.MrunForm

Parameter

strScriptName	Name des Scripts, das gestartet werden soll
mode	welcher Fenstertyp soll für das gestartete Script verwendet werden (siehe CWLScriptWindowType - Konstanten)

Rückgabewert (BOOL)

TRUE	Script konnte gestartet werden
FALSE	Script konnte nicht gestartet werden

SendMail(BSTR addr, BSTR strSubject, BSTR strText, BSTR attachments, BOOL bWithDialog)

Sendet eine Mail über MAPI32 mit dem am Computer eingerichteten Mailprofil.

Parameter

addr	Mailadresse
strSubject	Text für Betreff
strText	Text für Mailbody
attachments	Ein Dokument, das als Anhang versendet werden soll (mit Pfad)
bWithDialog	TRUE: Dialog wird angezeigt FALSE: Dialog wird nicht angezeigt

	(allerdings muß dann zumindest eine Mailadresse angegeben sein)
--	---

SetAppBackgroundPic (BSTR picture, int mode)

Diese Methode setzt das Hintergrundbild für das Applikationsfenster. Im Programm kann dies im Menüpunkt Parameter|Einstellungen im Reiter Design gemacht werden.

Der Parameter **mode** entspricht der Auswahl aus der dort vorhandenen Combobox ‚Darstellung‘.

Parameter

picture	Der Bildname (entweder als Dateiname mit Pfad, oder mit der Erweiterung .FROMDB um Bilder aus der Datenbank verwenden zu können.
mode	Art der Darstellung: 1... zentriert 2... gekachelt 3... Vollbild

SetAppBackgroundColor (short red, short green, short blue)

Diese Methode setzt die Hintergrundfarbe für das Applikationsfenster.

Die Parameter **red**, **green** und **blue** sind die Farbkomponenten, aus denen sich die Farbe zusammensetzt, jeweils ein Wert zwischen 0 und 255.

Parameter

red, green, blue	Die Farbkomponenten der Farbe jeweils zwischen 0 und 255, wobei 0,0,0 schwarz ist und 255,255,255 weiss.
------------------	--

void SetDefaultWinColor (short red, short green, short blue)

Diese Methode setzt die Standard-Hintergrundfarbe für alle Programmfenster.

Die Parameter **red**, **green** und **blue** sind die Farbkomponenten, aus denen sich die Farbe zusammensetzt, jeweils ein Wert zwischen 0 und 255.

Parameter

red, green, blue	Die Farbkomponenten der Farbe jeweils zwischen 0 und 255, wobei 0,0,0 schwarz ist und 255,255,255 weiss.
------------------	--

5.1.3. Events

OnQuit(ICWIEventResult *bResult)

Wird vor dem Beenden der Applikation gefeuert.

Soll das Beenden verhindert werden muß mit

bResult.Value = **False**

dies der Applikation mitgeteilt werden.

OnActivateApp(int AppNr)

Wird nach dem Umschalten auf eine Applikation gefeuert. Die aufgerufene Applikation kann aus dem Parameter **AppNr** entnommen werden (siehe dazu auch Konstanten - CWLApplicationNr).

OnWindowOpen(int AppNr, int windowId)

Wird gefeuert, wenn das Fenster mit der ID **windowId** in der Applikation mit der ID **AppNr** (siehe dazu auch Konstanten - CWLApplicationNr) geöffnet wird. Das Fenster ist zu diesem Zeitpunkt schon vorhanden. Dieser Event kann jedoch schon zu einem Zeitpunkt gefeuert werden, wo das betroffenen Fenster noch gar nicht dargestellt wird.

OnWindowClose(int AppNr, int windowId)

Wird gefeuert, wenn das Fenster mit der ID **windowId** in der Applikation mit der ID **AppNr** (siehe dazu auch Konstanten - CWLApplicationNr) geschlossen wird. Das Fenster ist zu diesem Zeitpunkt jedoch schon entladen und es kann darauf nicht mehr zugegriffen werden!

OnWindowActivate (CWLApplicationNr AppNr, int windowId);

Wird gefeuert, wenn ein Fenster aktiviert wird (z.B. mit der Maus angeklickt). Bei Script Fenstern geht es nur wenn sie vom Typ **cwlScriptWindowStandard** sind (bei CTK Fenster ist dies der Fenstertyp wenn das Fenster nicht modal definiert ist, wenn das Script mit **CWLStart. RunFormScript** gestartet wurde, muß der Parameter **mode** entsprechend gesetzt sein).

OnScriptWindowMayClose (CWLApplicationNr AppNr, int windowId, ICwlEventResult *bResult);

Wird gefeuert wenn das Skript Fenster geschlossen wird (bevor das Fenster tatsächlich geschlossen wird).
Mit
bResult.Value = [False](#)

kann das Schließen verhindert werden.

Der Parameter **windowId** ist nur gesetzt wenn das Fenster ein Standard CWL Fenster oder ein Script Fenster des Typs **cwlScriptWindowStandard** ist.

OnPageStarted (CWLApplicationNr AppNr, int windowId, int controlId, int PageNr, BSTR FormName)

Wird gefeuert, wenn beim Ausdruck eine neue Seite begonnen wird. Erfolgt die Ausgabe in eine Preview ist **controlId** die Id der Preview. **windowId** ist immer die ID jenes Fensters das den Ausdruck gestartet hat und nicht das Preview Fenster.

Ein möglicher Anwendungsfall:

Bevor der Ausdruck einer Seite gestartet wird, können bestimmte ausgedruckte Variablen noch geändert werden (über die Vars Eigenschaft des druckenden Fensters).

OnPagePrinted (CWLApplicationNr AppNr, int windowId, int controlId, int PageNr, BSTR FormName)

Wird gefeuert, wenn beim Ausdruck eine Seite fertig ausgegeben ist. Erfolgt die Ausgabe in eine Preview ist **controlId** die Id der Preview. **windowId** ist immer die ID jenes Fensters das den Ausdruck gestartet hat und nicht das Preview Fenster.

OnMessageBoxLaunched (CWLApplicationNr AppNr, int windowId, BSTR MessageBoxText, ICwlEventResult *ButtonPressed)

Wird gefeuert, wenn eine MessageBox angezeigt wurde (nachdem der Benutzer die MessageBox durch Drücken eines der MessageBox-Buttons diese geschlossen hat). Mit

ButtonPressed.Value = x

kann der tatsächlich gedrückte Button der MessageBox übersteuert werden (die Buttons werden von links nach rechts mit 1, 2, 3 .. usw. nummeriert).

**OnBeforeMessageBoxLaunched (CWLApplicationNr AppNr, int windowId, BSTR
MessageBoxText, ICwlEventResult *ButtonPressed)**

Wird gefeuert, bevor eine MessageBox angezeigt wird. Mit

ButtonPressed.Value = x

kann der tatsächlich gedrückte Button der MessageBox zurückgegeben werden was die Anzeige der MessageBox vollkommen unterdrückt (die Buttons werden von links nach rechts mit 1, 2, 3 .. usw. nummeriert). Wird -1 zurückgegeben, welches auch der Wert ist der automatisch gesetzt ist, dann wird die MessageBox angezeigt.

OnCompanyChange (BSTR CompanyNumber, int CompanyYear)

Wird gefeuert, wenn der Mandant gewechselt wird (auch wenn das Wirtschaftsjahr im Applikationstoolbar geändert wird, erfolgt ein Mandantenwechsel).

Im Parameter *CompanyYear* wird das Wirtschaftsjahr im internen numerischen Format übergeben (zur Konvertierung in das Textformat existieren im CWLCompany-Objekt Konvertierungsfunktionen).

5.2. CWLScript

Dieses eingebaute Objekt repräsentiert das Script selbst.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros
- ☐ Lohn Makros
- ☐ Fakt Makros

CWLScript
Eigenschaften
BSTR Name
ICwlWindow * CallingWindow
ICwlWindow * ScriptWindow
long ModalResult
Methoden
void Stop ()
void Hide ()
void Show ()
Events
OnScriptStart ()
OnScriptStop ()
OnParentClose ()

5.2.1. Eigenschaften

Name [BSTR, read only]

Name des Scripts. Siehe auch MacroCommands.MName.

CallingWindow [ICwlWindow*, read only]

Nur für CTK Scripts.

Referenz auf das Fenster Objekt (siehe Klasse CWLWindow), mit dem das CTK Script verknüpft ist. Das Fenster, das das Script darstellt kann mit der Eigenschaft ScriptWindow geholt werden.

ScriptWindow [ICwlWindow*, read only]

Gibt die Referenz auf das Script Fenster selbst zurück. Allerdings nur wenn das Skriptfenster vom Typ **cwlScriptWindowStandard** ist, was für CTK Fenster immer zutrifft (ausgenommen sie werden modal gestartet), für Systemscripts nur wenn sie mit dem entsprechenden Modus gestartet werden (siehe **RunFormScript**).

ModalResult [long, write only]

Mit dieser Eigenschaft wird das Ergebnis des Aufrufs eines modalen Fensters gesetzt.

5.2.2. Methoden

Stop

Entladet das Script (keine Events werden mehr behandelt). Nicht zu verwechseln mit dem VBScript **Stop** befehl.

Danach muss das laufende Script unbedingt mit der nächsten Anweisung mit **Exit Sub** oder **Exit Function** beendet werden.

Hide

Versteckt das Fenster, das mit dem Script verbunden ist.

Show

Zeigt ein verstecktes Script-Fenster wieder an.

5.2.3. Events

OnScriptStart

Wird gefeuert, wenn das Script gestartet wird.

OnScriptStop

Wird gefeuert, wenn das Script beendet wird.

OnParentClose

Wird nur bei CTK Scripts gefeuert, wenn das mit dem Script verknüpfte Fenster geschlossen wird.

5.3. CWLCurrentModule

Dieses Objekt dient nur dazu Events für das aktuell aktive Modul auszuwerten. Das Objekt kann nur im Rahmen der CTK Scripts verwendet werden.

CWLCurrentModule
Eigenschaften
ICwlModule* ActiveModule
Events
OnActivate (int AppNr)
OnWindowOpen (int windowId)
OnWindowClose (int windowId)

Verfügbar in

☐ CTK Makros

5.3.1. Eigenschaften

ActiveModule [ICwlModule*, read only]

Liefert einen Pointer auf das aktuell aktive Modul.

5.3.2. Events

Diese Events werden nur bei CTK Scripts an das entsprechende Fensterscript gesendet. Systemscripts erhalten diese Events nicht. Die Events werden nur an die im aktuellen Module aktiven CTK Script Fenster gesendet.

OnActivate(int AppNr)

Wird gefeuert, wenn auf das Modul umgeschaltet wird.

OnWindowOpen (int windowId)

Wird gefeuert wenn das Fenster mit der ID **windowId** geöffnet wird.

OnWindowClose (int windowId)

Wird gefeuert wenn das Fenster mit der ID **windowId** geschlossen wird.

5.4. CWLCurrentWindow

Dieses Objekt dient nur dazu Events für das aktuell aktive Fenster auszuwerten. Das Objekt kann nur im Rahmen der CTK Scripts verwendet werden. Die Events

Verfügbar in

☐ CTK Makros

CWLCurrentWindow

Eigenschaften

ICwlModule* **ActiveWindow**

Events

OnActivate (int nWinId)

OnControlActivate (int nFgld)

OnCheck (int nFgld)

OnBeforeCheck (int nFgld, BSTR Contents)

OnGridCheck (int nFgld)

OnGridChangeLine (int nFgld)

OnPushButton (int nFgld, ICwlEventResult *bResult)

OnCheckBox (int nFgld)

OnRadioButton (int nFgld)

OnChangeButton (int nFgld)

OnCheckUserfield (int nFgld, ICwlEventResult *bResult)

OnChangeFilter (BSTR FilterName, ICwlEventResult *bResult)

OnChangeCompanyYear (int CompanyYear, ICwlEventResult *bResult)

OnChangeCompany (const char *company, int CompanyYear, ICwlEventResult *bResult)

OnGridDbiClick (int nFgld, ICwlEventResult *bResult)

OnDynamicMenuCommand (int nFgId, int MenuIndex, ICwlEventResult *bResult)
OnAfterEvent (int nFgId, int EventType, int Originalresult)
OnGridCheckUserColumn (int nFgId, int row, int column, ICwlEventResult *bResult)
OnGridNewUserLine (int nFgId, int row, int column, ICwlEventResult *bResult)
OnSearch (int nFgId, ICwlEventResult *bResult)
OnGridSearch (int nFgId, int Zeile, int Spalte, ICwlEventResult *bResult)
OnGridCheckBox (int nFgId, int Zeile, int Spalte)
OnGridDrillDown (int nFgId, int Zeile, int Spalte, ICwlEventResult *bResult)
OnCmbSelChange (int nFgId, ICwlEventResult *bResult)
OnGridCmbSelChange (int nFgId, int row, int column, ICwlEventResult *bResult)
OnGridAllowEdit (nFgId, nRow, nColumn, bResult)
OnUserEvent (int EventType, VARIANT *Data, ICwlEventResult *bResult)

5.4.1. Eigenschaften

ActiveWindow [ICwlWindow*, read only]

Liefert einen Pointer auf das aktuell aktive Fenster.

5.4.2. Events

Diese Events werden nur bei CTK Scripts an das entsprechende Fensterscript gesendet. Das bedeutet, daß bei einem mit dem CTK geänderten Fenster Elemente als Eventauslöser vom Anwender definiert werden (Buttons, Editfelder... usw.), und das mit diesem Fenster verknüpfte Script empfängt dann diese Events.

void OnActivate(int nWinId)

Wird gefeuert, wenn das Fenster aktiviert wird.

OnControlActivate(int nFgId)

Wird gefeuert wenn das Element mit der ID **nFgId** den Focus erhält.

OnCheck(int nFgId)

Wird gefeuert wenn ein Editfeld oder eine Combobox verlassen wird, nachdem die Applikation die Eingabe geprüft hat. Wird von der Applikation das Verlassen des Feldes nicht gestattet (z.B. bei fehlerhafter Eingabe) wird dieses Event nicht gefeuert.

OnBeforeCheck(int nFgId, BSTR Contents, ICwlEventResult *bResult)

Wird gefeuert wenn ein Editfeld oder eine Combobox verlassen wird, bevor die Applikation die Eingabe geprüft hat. In 'Contents' wird der eingegebene Text im internen Format übergeben (ein Datum hat z.B. das Format 'tt-mm-jjjj', eine Fließkommazahl einen Punkt als Dezimaltrenner '3.14').

Wird das Ergebnis auf FALSE gesetzt (bResult.value = FALSE), kann das Feld nicht verlassen werden. Wird der Wert auf eine Feldnummer (bResult.value = 112) gesetzt, dann wird das Feld verlassen ohne dass die Anwendung eine Meldung erhält und direkt zu dem Feld gesprungen.

Wird das Ergebnis nicht verändert, bzw. mit auf bResult.value = TRUE gesetzt, dann erhält die Anwendung als nächstes die MSG_CHECK so als ob nichts passiert wäre.

OnGridCheck(int nFgId)

Wird gefeuert, wenn in einem Grid eine Zelle mit einem Editfeld oder einer Combobox verlassen wird. Wird von der Applikation das Verlassen der Zelle nicht gestattet (z.B. bei fehlerhafter Eingabe) wird dieses Event nicht gefeuert.

OnGridCheck(int nFgId, ICwlEventResult *bResult)

Wird gefeuert, wenn in einem Grid eine Zelle mit einem Editfeld oder oder einer Combobox verlassen wird. Das Script erhält die Auswertemöglichkeit vor der Applikation und kann durch Setzen von `bResult.Value=false` verhindern, dass die Zelle verlassen werden kann.

OnGridChangeLine(int nFgId)

Wird gefeuert, wenn in einem Grid die Zeile gewechselt wird.

OnPushButton(int nFgId, ICwlEventResult *bResult)

Wird gefeuert, wenn ein Button gedrückt wird. Das Script erhält die Auswertemöglichkeit vor der Applikation und kann durch Setzen von `bResult.Value = false` verhindern, daß die Applikation das Drücken des Buttons erfährt.

OnCheckBox(int nFgId)

Wird gefeuert, wenn eine Checkbox angeklickt wird.

OnRadioButton(int nFgId)

Wird gefeuert, wenn eine Radiobutton - Gruppe verlasen wird.

OnChangeButton(int nFgId)

Wird gefeuert, wenn innerhalb einer Radiobutton - Gruppe der selektierte Button geändert wird.

OnCheckUserfield (int nFgId, ICwlEventResult *bResult)

Wird gefeuert, wenn ein vom Benutzer im CTK angelegtes Editfeld oder Combobox verlassen wird. Mit `bResult.Value = False`

kann das Verlassen des Feldes verhindert werden.

Nach dem Event wird der eingegebene Wert automatisch in die zugehörige Variable kopiert (vorausgesetzt `bResult.Value = False` wird nicht gesetzt).

Im Event muss der aktuelle Wert aus dem Eingabefeld mit der Property *ScreenContents* geholt werden, die Property *Contents* enthält noch den ursprünglichen Wert (wie auch die mit dem Feld verbundene Variable).

OnChangeFilter (BSTR FilterName, ICwlEventResult *bResult)

Wird gefeuert, wenn in der Filter-Combobox eines Fensters der Filter gewechselt wird. Mit `bResult.Value = False`

wird verhindert, dass der Wechsel tatsächlich durchgeführt wird. Im Parameter *FilterName* wird der Name des Filters übergeben.

OnChangeCompanyYear (int CompanyYear, ICwlEventResult *bResult)

Wird gefeuert, wenn in der Wirtschaftsjahr-Combobox im Applikationstoolbar das aktuelle Wirtschaftsjahr gewechselt wird. Mit

`bResult.Value = False`

wird verhindert, dass der Wechsel tatsächlich durchgeführt wird. Im Parameter *CompanyYear* wird das Wirtschaftsjahr im internen numerischen Format übergeben (zur Konvertierung in das Textformat existieren im CWLCompany-Objekt Konvertierungsfunktionen).

OnChangeCompany (const char *Company, int CompanyYear, ICwlEventResult *bResult)

Wird gefeuert, wenn in der Applikation der Mandant (bzw. das WJ) gewechselt wird. Mit

bResult.Value = **false**

wird verhindert, dass der Wechsel tatsächlich durchgeführt wird. Im Parameter *CompanyYear* wird das Wirtschaftsjahr im internen numerischen Format übergeben (zur Konvertierung in das Textformat existieren im CWLCompany-Objekt Konvertierungsfunktionen).

Im Normalfall kann dieses Event nicht benützt werden, weil während des Mandantenwechsel in der Regel keine Fenster offen sein können (eine Ausnahme ist das Cockpitfenster).

OnGridDbClick (int nFgId, ICwlEventResult *bResult)

Wird gefeuert, wenn in einer Bildschirmtablette in einer nicht veränderbaren Spalte doppelt geklickt (oder die ENTER – Taste gedrückt wird. Das Script erhält die Auswertemöglichkeit vor der Applikation und kann durch Setzen von bResult.Value = **false**

verhindern, daß die Applikation die mit dem Click/Tastendruck verknüpfte Aktion durchführt.

OnDynamicMenuCommand (int nFgId, int MenuIndex, ICwlEventResult *bResult)

Wird gefeuert, wenn in einem Fenster mit einem Auswahlbutton (z.B. Drucker/Bildschirm) eine Auswahl getroffen wird. Dies erfolgt auch, wenn der entsprechende Button mit F5 ausgewählt wird. Der ausgewählte Menüpunkt des Buttons wird im MenuIndex übergeben, wobei der erste Menüpunkt den Index 0 hat und die weiteren aufsteigend nummeriert sind.

Das Script erhält die Auswertemöglichkeit vor der Applikation und kann durch Setzen von bResult.Value = **false**

verhindern, dass die Applikation die mit dem Click/Tastendruck verknüpfte Aktion durchführt.

OnAfterEvent (int nFgId, int EventType, int Originalresult)

Wird *nach* der Abarbeitung bestimmter Events gefeuert. Damit kann auf Ereignisse in der CWL reagiert werden, nachdem diese ausgeführt wurden (z.B. Buttondruck).

Für die folgenden Eventtypen (Parameter *EventType*) wird das Event gefeuert:

- ☐ Pushbutton (EventType = 7)
- ☐ Checkbox (EventType = 5)
- ☐ Radiobutton (EventType = 6)
- ☐ Radiobutton - Change-Event (EventType = 10)
- ☐ Listboxauswahl (EventType = 4)
- ☐ Combobox Selektionsänderung (EventType = 81)
- ☐ Doppelklick in Tabelle (EventType = 30)
- ☐ Checkbox in einer Tabelle (EventType = 26)
- ☐ CheckMsg in einer Tabelle (EventType = 22)
- ☐ ChangeLine in einer Tabelle (EventType = 27)
- ☐ Neue Zeile in einer Tabelle (EventType = 29)
- ☐ Wechsel in eine Zelle in einer Tabelle (EventType = 21)
- ☐ Combobox Selektionsänderung in einer Tabelle (EventType = 82)
- ☐ Tree Doppelklick (EventType = 96)
- ☐ Tree Selektionsänderung (EventType = 98)
- ☐ Tree Eintrag löschen (EventType = 97)
- ☐ Fenster Startup (EventType = 1)

Der *OriginalResult* - Wert beinhaltet das Ergebnis des Events aus der CWL (im Regelfall ist dies 0, zumeist wird der Wert -1 als Fehlerbedingung verwendet, die genaue Bedeutung hängt vom Anwendungsfall ab).

OnGridCheckUserColumn(int nFgId, int Row, int Column, ICwlEventResult* bResult)

Wird gefeuert, wenn in einer Bildschirmtabelle in einer hinzugefügten Spalte eine Zelle mit einem Editfeld oder einer Combobox verlassen wird.

Das Setzen von `bResult.Value = false` verhindert, dass die Zelle verlassen werden kann.

OnGridNewUserLine(int nFgId, int Row, int Column, ICwlEventResult* bResult)

Wird gefeuert, wenn in einer Bildschirmtabelle in die leere Zeile nach der letzten Zeile gewechselt wird. Mit dem `bResult.Value = false` wird die weitere Eventverarbeitung unterbrochen, was bei diesem Event aber keine praktische Auswirkung hat. Das Event wird nur bei selbst definierten Grids gefeuert und kann dazu verwendet werden, beim Wechseln in eine neue Zeile automatisch eine zusätzliche Zeile einzufügen.

OnSearch (int nFgId, ICwlEventResult *bResult)

Wird gefeuert, wenn in einem Editfeld auf die Lupe geklickt wird, oder die Taste F9 gedrückt wird. Falls das Element nicht vom Benutzer angelegt wurde, kann mit `bResult.value = false` das Standardverhalten unterbunden werden (der Matchcode wird dann vom Programm nicht aufgerufen).

OnGridSearch (int nFgId, int Zeile, int Spalte, ICwlEventResult *bResult)

Wird gefeuert, wenn in einem Editfeld einer Bildschirmtabelle auf die Lupe geklickt wird, oder die Taste F9 gedrückt wird.

Falls das die Bildschirmtabelle nicht vom Benutzer angelegt wurde, kann mit `bResult.value = false` das Standardverhalten unterbunden werden (der Matchcode wird dann vom Programm nicht aufgerufen).

OnGridCheckBox(int nFgId, int Zeile, int Spalte)

Wird gefeuert, wenn in einem Grid in eine Zelle mit Checkbox geklickt wird, oder innerhalb einer solchen Zelle mit der Leertaste der Status der Checkbox umgestellt wird.

OnGridDrillDown(int nFgId, int Zeile, int Spalte, ICwlEventResult *bResult)

Wird gefeuert, wenn in einem Grid in einer Zelle auf ein Drilldown geklickt wird. Das Event wird nur ausgelöst, wenn das Drilldown nicht einen vorgegebenen Objekttyp hinterlegt hat.

Mit `bResult.Value = false` kann das Defaultverhalten der Anwendung unterbunden werden.

OnCmbSelchange (int nFgId, ICwlEventResult *bResult)

Wird gefeuert, wenn in einer Combobox die Selektion verändert wird, ohne dass die Combo verlassen wird. Mit `bResult.Value = false` kann das Defaultverhalten der Anwendung unterbunden werden.

Um den aktuell selektierten Wert zu bekommen muss die *ScreenContents* – Eigenschaft des Controls verwendet werden. Diese beinhaltet immer den gesamten Text des Combobox-Eintrags.

OnGridCmbSelchange (int nFgId, int Zeile, int Spalte, ICwlEventResult *bResult)

Wird gefeuert, wenn in einem Grid in einer Zelle mit einer Combobox ein anderer Eintrag der Combobox ausgewählt wird, ohne die Zelle dabei zu verlassen.

Mit `bResult.Value = false` kann das Defaultverhalten der Anwendung unterbunden werden.

Um den aktuell selektierten Wert zu bekommen muss die *ScreenContents* – Eigenschaft des Controls verwendet werden. Diese beinhaltet immer den Inhalt des Listenindex des Combobox-Eintrags.

OnGridAllowEdit (nFgId, nRow, nColumn, bResult)

Dieses Event wird bei jeder Zelle in der Grid gefeuert, bevor das Editfeld/Checkbox/Combo in der Zelle aktiviert wird. Mit `bResult.Value = false` kann nicht mehr bearbeitet werden (read only).

OnUserEvent (int EventType, VARIANT Data, ICwlEventResult *bResult)

Wird im Fenster gefeuert, wenn die *SendWindowEvent* - Methode in *CWLModule* für dieses Fenster aufgerufen wird.

In *EventType* wird ein beliebiger numerischer Wert übergeben, der das Event beschreibt und in *Data* wird ein beliebiger zusätzlicher Wert übergeben, der auch ein Array von Werten sein kann. Beide Parameter werden in der *SendWindowEvent* - Methode gesetzt.

In *bResult* kann ein beliebiger numerischer Wert als Ergebnis zurückgeliefert werden (`bResult.Value = 42`), der als Rückgabewert der *SendWindowEvent* - Methode verwendet wird.

5.5. CWLWindowVars

Dieses Objekt dient zum Zugriff auf die Variablen eines Fensters. Mit dem Objekt kann direkt auf alle Variablen zugegriffen werden, die innerhalb des Fensters von der Applikation definiert sind.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLWindowVars**Eigenschaften**

VARIANT **Value** (short nView, short nVar)

VARIANT **UserValue** (short nView, VARIANT Var)

VARIANT_BOOL **Locked** (short nView, short nVar)

Methoden

BOOL **CreateVar** (short nView, short nVar, BSTR Type, int length, VARIANT Value, VARIANT bOverwriteExisting = FALSE)

5.5.1. Eigenschaften

Value (short nView, short nVar) [VARIANT, read write]

Mit **nView** und **nVar** wird die gewünschte Variable ausgewählt. **nView** kann einerseits die der Variable zugrundeliegende Tabellenummer sein (z.B. Mandantenstamm → Tabelle T001 → `nView = 1`) oder auch 0 sein. Variablen mit `nView = 0` sind Variablen, die vom Programm angelegt werden und mit keiner Datenbanktabelle im Zusammenhang stehen.

nVar ist, bei Datenbanktabellenvariablen die Spaltennummer (z.B. Mandantenstamm Straße → Spalte C004 → `nVar = 4`), wenn `nView = 0` ist es eine Nummer die bei 20 beginnt. Nummern < 20 haben in allen Fenstern die gleiche Bedeutung:

Nr	Bedeutung
11	MandantNr
12	Pfad
13	MandantName
14	BenutzerName
15	Version
16	Auswertedatum
17	MailAdresse
18	Datenstandsversion

Die Variablen ≥ 20 hängen vom Fenster ab und entsprechen im Regelfall den Variablen wie sie im CWLPDFE zur Verfügung stehen.

UserValue (short nView, VARIANT Var) [VARIANT, read write]

Für den Zugriff auf benutzerdefinierte Spalten in einer Mandantentabelle kann mit dieser Eigenschaft auf diese Spalten zugegriffen werden (die erste benutzerdefinierte Spalte einer Tabelle: U000 wird mit UserValue (nView, 0) oder userValue(nView, "U000") oder mit UserValue (nView, "Spaltenname") angesprochen). Die benutzerdefinierte Spalten werden immer ab der Variable 500 eingefügt (das obige Beispiel würde damit auch mit Value(nView, 500) funktionieren).

Bei benutzerdefinierten Tabellen werden die Uxxx - Spaltenwerte auf die Variable xxx gemappt, damit ist dort der Zugriff sowohl mit Value als auch UserValue mit den gleichen Parametern möglich (ausgenommen der definierte Spaltenname, der nur bei der UserValue - Eigenschaft möglich ist).

Parameter

nView	Die Nummer der Tabelle, für die die Variablen angelegt sind
Var	Der Index der UserVar (0 für U000), oder der Name der Spalte selbst ("U000") oder der definierte Name ("Name")

Locked (short nView, short nVar) [VARIANT BOOL, read write]

Mit dieser Eigenschaft kann der 'Gesperrt' - Status einer Variable ausgelesen oder verändert werden. Variablen, die sich auf eine Tabelle beziehen haben diesen Status bereits abhängig von den Benutzerberechtigungen gesetzt. Variablen, die vom Programm angelegt werden (nView = 0) haben dies in der Regel nicht und können nun gesperrt werden, damit sie z.B. in Formularen nur als ***** dargestellt werden.

Mit **nView** und **nVar** wird die gewünschte Variable ausgewählt. **nView** kann einerseits die der Variable zugrundeliegende Tabellenummer sein (z.B. Mandantenstamm → Tabelle T001 → nView = 1) oder auch 0 sein. Variablen mit nView = 0 sind Variablen, die vom Programm angelegt werden und mit keiner Datenbanktabelle im Zusammenhang stehen.

Parameter

nView	Die Nummer der Tabelle, für die die Variablen angelegt sind oder 0
Var	Die Nummer der Variable

5.5.3. Methoden

BOOL CreateVar (short nView, short nVar, BSTR Type, int length, VARIANT Value, VARIANT bOverwriteExisting)

Die Funktion erzeugt eine neue Variable mit einer bestimmten Nummer innerhalb der angegebenen View. Existiert bereits eine Variable mit dieser Nummer, wird FALSE zurückgegeben. Soll eine existierende Variable automatisch ersetzt werden, muss für **bOverwriteExisting** TRUE übergeben werden. FALSE kann auch zurückgegeben werden, wenn der übergebene Wert in **Value** nicht auf den Typen der Variable konvertiert werden kann (z.B. ein ungültiges Datum).

Mit **nVar** wird die Nummer der anzulegenden Variable angegeben. Die Nummer kann mit 0 beginnen und kann maximal 1000 Einträge erhalten. Die Nummern müssen nicht aufsteigend vergeben werden.

Mit **Type** wird der Typ der Variablen festgelegt, wobei die folgenden Typen möglich sind:

Type	Bedeutung
1	Textvariable (Länge wählbar)
2	Zahl ohne Nachkommastellen (Integer)
4	Zahl mit Nachkommastellen (Double)
6	Datum mit Zeit

Mit **length** wird die Länge von Textvariablen vorgegeben. Bei allen anderen Typen kann hier 0 übergeben werden.

In **Value** kann ein Wert übergeben werden, der als der Startwert der Variable eingetragen wird. Dieser Parameter ist optional und kann auch weggelassen werden.

Wird **bOverwriteExistings** mit TRUE übergeben, können bereits existierende Variablen ersetzt werden.

5.6. CWLEventResult

Dieses Objekt dient bei Events, die ein Ergebnis erwarten, als Rückgabewert. Durch die speziellen Gegebenheiten in VBScript kann nicht der Wert direkt gesetzt werden, sondern es muss immer die Syntax `result.Value = true` (oder `false`) verwendet werden.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLEventResult
Eigenschaften
BOOL Value

5.6.1. Eigenschaften

Value [BOOL, read write]

Diese Eigenschaft nimmt das Ergebnis des Events auf.

Beispiel

```

Sub CWLStart_OnScriptWindowMayClose(AppNr, windowId, bResult)
    If <Bedingung> Then
        bResult.Value = True
    Else
        bResult.Value = False
    End If
End Sub

```

5.7. CWLSearchResult

Dieses Objekt dient zum Zugriff auf die Ergebnisse einer SQL Abfrage, wie sie im Objekt CWLCompany durchgeführt werden kann. Abhängig von der Abfrage stehen die Ergebnisspalten, die Namen dieser Spalten und die Anzahl zur Verfügung.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLSearchResult
Eigenschaften
short MaxColumnIndex
int RowCount
VARIANT Value (VARIANT ColumnIndexOrName)
BSTR ColumnName (short nColumnIndex)
Methoden
BOOL NextRecord ()
Close ()
CopyResultsToWindow (short WindowId, VARIANT View)

5.7.1. Eigenschaften**Value(VARIANT IndexOrName) [VARIANT, read only]**

Gibt den Ergebniswert mit dem entsprechenden Index (oder dem entspr. Spaltennamen) zurück.

MaxColumnIndex [short, read only]

Gibt den Index der letzten Ergebnisspalte zurück (entspricht Anzahl der Ergebnisspalten - 1).

ColumnName (short Index) [BSTR, read only]

Gibt den Namen der Spalte mit der Nummer **Index** zurück, wobei Index zwischen 0 und **MaxColumnIndex** liegen kann.

RowCount [int, read only]

Gibt die Anzahl der enthaltenen Datensätze zurück.

Beispiel

```

Dim text, result, i
edtResult = "" ' Textfeld für das Ergebnis

```

On Error Resume Next

```
' Suche in der Tabelle T024 nach dem Artikel ,10001'
' Ab der Version 8.0 muss in der Abfrage auch der Mandant und das Wirtschaftsjahr eingefügt werden!
' dabei wird der aktuelle Mandant durch '~~~~' eingefügt und das aktuelle Wirtschaftsjahr mit yyyy
' das Ergebnis ist ein CWLSearchResult - Objekt dessen Default Property
' MaxColumnIndex ist, welcher -1 ist, wenn nichts gefunden wurde
Set result = CWLStart.CurrentCompany.SearchRecord ("T024", "C002 = '10001' and MESOCOMP = '~~~~'
and MESOYEAR = yyyy")
If result < 0 Then
    If err <> 0 Then
        ' Fehler aufgetreten (z.B. C002 ist keine gültige Spalte)
        MsgBox err.description
        Exit Sub
    Else
        ' Artikel nicht gefunden
        MsgBox "Could not find the requested record."
        Exit Sub
    End If
End If

' alle Spalten werden mit Spaltenname und Inhalt in einem Textfeld
' angezeigt.
i = 0
For i = 0 To result.MaxColumnIndex
    text = result.ColumnName (i) & ": " & result.Value (i) & chr (13)
    If err <> 0 Then
        MsgBox err.description
        Exit Sub
    End If
    edtResult = edtResult + text
Next

' Ergebnis der Routine im Textfeld edtResult
' C002: 10001
' C003: Rennrad 26 "
' C011: 10001
' C014: 0
' C020: 0
' usw.
```

5.7.2. Methoden

NextRecord()

Rückgabewert (BOOL)

Liefert True zurück wenn ein weiterer Datensatz gelesen wurde, oder False, wenn keine Datensätze mehr zur Verfügung stehen.

Close

Schließt das Objekt und verwirft alle darin gespeicherten Daten.

CopyResultsToWindow (short WindowId, VARIANT View)

Es werden die aktuellen Werte des Objekts in die Tabellenvariablen des angegebenen Fensters kopiert. Die Anzahl und der Typ der Variablen in der Zieltabelle muss exakt den Werten im Objekt entsprechen. Nur wenn als Zieltabelle 495 (Benutzervariable) verwendet wird, werden die Variablen im Fenster neu angelegt oder mit dem neuen Typ überschrieben.

Parameter

WindowId	Die Nummer des Fensters, in das die Werte kopiert werden
View	Die Tabellennummer der Zielvariablen (wenn leer wird 495 verwendet)

5.8. GeneralScriptFuncs

Dieses Objekt öffnet Fenster des Betriebssystems (bzw. der VBScript Engine) auf dem Client Computer, die im Falle der EWL auf dem Server geöffnet würden.

MsgBox und *InputBox* sind Funktionen der VBScript Engine (in der CWL können direkt die VBScript Funktionen verwendet werden, wenn die Kompatibilität zur EWL keine Rolle spielt), *FileDialog* öffnet den Dateiauswahldialog des Betriebssystems.

Verfügbar in

☐ überall

GeneralScriptFuncs
Eigenschaften
BOOL WaitCursor
Methoden
int MsgBox (BSTR prompt, VARIANT buttons, VARIANT title)
BSTR InputBox (BSTR prompt, VARIANT default, VARIANT title)
BSTR FileDialog (BOOL Open, BSTR Extension, BSTR Filename, VARIANT Filter)
BSTR Convert (BSTR Input, VARIANT ConvertTo)
MsgWin (BSTR Title, VARIANT bMitAbbruchButton)
BOOL MsgWinSetText (BSTR Text1, BSTR Text2)
MsgWinDestroy ()

5.8.1. Eigenschaften

WaitCursor [BOOL, read write]

Mit dieser Eigenschaft kann statt dem Mauscursor die Sanduhr angezeigt werden.

5.8.2. Methoden

int MsgBox (BSTR prompt, VARIANT buttons, VARIANT title)

Die Funktion entspricht der VBScript Funktion MsgBox.

BSTR InputBox (BSTR prompt, VARIANT default, VARIANT title)

Die Funktion entspricht der VBScript Funktion InputBox, nur sind die Parameter default und title vertauscht.

BSTR FileDialog (BOOL Open, BSTR Extension, BSTR Filename, VARIANT Filter)

Mit dieser Funktion kann der Dateiauswahldialog des Betriebssystems geöffnet werden.

Ist **Open** auf TRUE gesetzt, wird der Öffnendialog angezeigt, andernfalls der Speicherndialog.

In **Extension** kann eine Dateierweiterung übergeben werden, die verwendet wird, wenn der Anwender eine Datei ohne Erweiterung eingibt.

In **Filename** kann ein Dateiname inklusive Pfad angegeben werden, der zu Beginn in der Eingabe angezeigt wird.

Mit **Filter** (optional) kann im Dateidialog die Typauswahlbox mit zulässigen Datentypen gefüllt werden.

Wird hier nichts übergeben wird "Alle Dateien (*.*)" verwendet. Die einzelnen Dateitypen werden mit dem '|' - Zeichen voneinander getrennt, das Ende wird mit zwei '||' gekennzeichnet ("Textdateien (*.txt)|*.txt|Spooldateien (*.spl)|*.spl||").

Wird eine Datei ausgewählt, wird der gewählte Dateiname inklusive Pfad von der Funktion zurückgegeben. Bei Abbruch mit ESC wird ein Leerstring zurückgegeben.

Beispiel

```
general.msgbox general.FileDialog (True, "txt", "c:\temp\scripttest\*.txt", "Textdateien (*.txt)|*.txt|Spooldateien (*.spl)|*.spl||")
```

BSTR Convert (BSTR Input, VARIANT ConvertTo)

Mit dieser Funktion kann ein Text (**Input**) in das Base64 - Format konvertiert werden.

Der Parameter **ConvertTo** beschreibt das Format in das konvertiert werden soll:

ConvertTo = 0: von Unicode/UTF8 nach Base64

Der übergebene Text wird zuerst nach UTF8 konvertiert und der daraus resultierende Text nach Base64.

Das Base64 - Format ist eigentlich ein reines ASCII - Format. Bei der Rückgabe wird es als BSTR zurückgegeben, das UNICODE enthält, was bedeutet dass der resultierende UNICODE-Text trotzdem reines ASCII enthält.

ConvertTo = 1: von Base64 nach Unicode/UTF8

Der übergebene Text muss Base64 - Code sein (reines Ascii) welcher dekodiert wird. Das Ergebnis wird als UTF8 interpretiert, und daraus wird wieder Unicode erzeugt.

ConvertTo = 2: von RTF Text nach Plain Text

Wenn der übergebene Text im RTF-Format ist, wird er in Text ohne Konvertierungen formatiert. Ist der Text bereits ohne Formatierungen, macht die Funktion nichts mit dem Text.

Beispiel 1:

```
Text = "[BELEGNR] = " & Value(25,43) & " [JAHR] = " & Value(0,5) & " [MANDANT] = " & Value(0,11)
ResultValue = General.Convert (Text)
```

In ResultValue wird dann

```
"W0JFTEVHTIJdID0gQU4xNC01MTMgW0pBSFJdID0gMjAxNCBbTUFOREFOVF0gPSAzMDBNAA=="
zurückgegeben.
```

Beispiel 2:

```
Text = "W0JFTEVHTIJdID0gQU4xNC01MTMgW0pBSFJdID0gMjAxNCBbTUFOREFOVF0gPSAzMDBNAA=="
ResultValue = General.Convert (Text, 1)
```

In ResultValue wird dann der in Beispiel 1 generierte String z.B. "[BELEGNR] = AN14-513 [JAHR] = 2014 [MANDANT] = 300M" gestellt.

Beispiel 3:

```
Text = "{\rtf1\ansi\ansicpg1252\deff0\nouicompat\deflang1031{\fonttbl{\f0\fswiss\fcharset0 Arial;}}
{\*\generator Riched20 10.0.10240}\viewkind4\uc1
\pard\nowidctlpar\hyphpar0\charscalex100\b\f0\fs20 Test\par}"
ResultValue = General.Convert (Text, 2)
```

In ResultValue wird dann der "Test" gestellt.

MsgWin (BSTR Title, VARIANT bMitAbbruchButton)

Mit dieser Funktion kann während einer länger dauernden Schleife innerhalb des Scripts ein kleines Fenster mit einem Text dargestellt werden. Optional kann das Fenster auch einen Abbruchbutton enthalten, mit dem der Benutzer die Schleife abbrechen kann.

Parameter

Title	Der Titel des Fensters
bMitAbbruchButton	Ob ein Abbruchbutton angezeigt werden soll (optional, default ist FALSE)

BOOL MsgWinSetText (BSTR Text1, BSTR Text2)

Mit dieser Funktion kann der dargestellte Text bei jedem Durchlauf verändert werden. Mit der Funktion wird auch geprüft, ob der Benutzer bei angezeigtem Abbruch-Button diesen gedrückt hat.

Parameter

Text1	Text der ersten Zeile innerhalb des Fensters
Text2	Text der zweiten Zeile (kann leer bleiben)

Rückgabe: FALSE wenn das Fenster durch einen Druck auf den Abbruchbutton geschlossen werden soll, sonst TRUE.

MsgWinDestroy ()

Mit dieser Funktion wird das Fenster wieder geschlossen.

Beispiel

```
General.MsgWin "Ich zähle bis 30", True ' mit Abbruchbutton

For i = 1 To 30
    If General.MsgWinSetText ("erste Zeile", "zweite Zeile: " & i) = False
Then
        Exit For
    End If
    MacroCommands.MWait 1000 ' Wartet 1 Sekunde
Next

General.MsgWinDestroy
```


5.9. CWLTable

Dieses Objekt stellt eine geöffnete Datenbanktabelle dar, mit dem Werte aus der Tabelle gelesen werden können, neue Werte eingefügt werden können, bestehende Werte aktualisiert werden könne und Datensätze gelöscht werden können. Diese Basisfunktionalität funktioniert auf Datensatzebene (es wird immer ein Datensatz verändert, der mit einem eindeutigen Schlüssel gefunden werden kann).

Jede Spalte der Tabelle wird als Variable im zugehörigen Fenster angelegt, damit können die Tabelleneinhalte direkt im Fenster oder in PDFs verwendet werden.

Zusätzlich kann ein SQL Ausdruck auf die Tabelle abgesetzt werden, wo die gefundenen Werte ebenfalls in die Tabellenvariablen geschrieben werden und diese damit ebenfalls im Fenster oder in PDFs zur Verfügung stehen.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLTable
Eigenschaften BSTR Name BOOL Valid int MaxColIndex Methoden VARIANT Value (VARIANT column) void Value (VARIANT column, VARIANT newValue) BOOL Get (BSTR Key, VARIANT ExpandKey) BOOL Update () BOOL Insert () BOOL Delete (BSTR Key, VARIANT WhereStmt) void CopyToWindow (short Window) CWLSearchResult * Select (BSTR SelectStmt)

5.9.1. Eigenschaften

Name (BOOL, read only)

In dieser Eigenschaft ist der Name der Tabelle hinterlegt.

Valid (BOOL, read only)

In dieser Eigenschaft ist hinterlegt, ob die Tabelle erfolgreich geöffnet wurde.

MaxColIndex (int, read only)

In dieser Eigenschaft ist der Index der letzten definierten Spalte hinterlegt (entspricht damit der letzten für diese Tabelle angelegten Variable).

5.9.2. Methoden

VARIANT Value (VARIANT column)

Mit dieser Methode kann auf die Variablen der Tabelle (die Spaltenwerte) zugegriffen werden.

Parameter

column	Der Index der Variable oder der Spaltenname. Bei benutzerdefinierten Tabellen/Spalten, kann auch der dort hinterlegte Spaltenname verwendet werden (dieser ist aber sprachabhängig, und könnte bei mehrsprachigen Anwendungen zu Problemen führen)
--------	--

Rückgabewert: der Spaltenwert

void Value (VARIANT column, VARIANT newValue)

Mit dieser Methode kann ein Spaltenwert verändert werden.

Parameter

column	Der Index der Variable oder der Spaltenname. Bei benutzerdefinierten Tabellen/Spalten, kann auch der dort hinterlegte Spaltenname verwendet werden (dieser ist aber sprachabhängig, und könnte bei mehrsprachigen Anwendungen zu Problemen führen)
newValue	der neue Wert für die Variable (Spalte)

Beispiel

```
if table.value(1) > 100 then
    table.value(1) = table.value(1)/100
end if
```

BOOL Get (BSTR Key, VARIANT ExpandKey)

Mit dieser Methode kann der Datensatz der Tabelle mit dem eindeutigen Schlüssel 'Key' gelesen werden. Wird der Datensatz gefunden, werden die Tabellenvariablen mit dem Inhalt des Datensatzes gefüllt.

Parameter

Key	Der eindeutige Schlüsselwert, der in der beim OpenTable angegebenen Spalte drinsteht
ExpandKey	Bei mesonic Datenstandtabellen, muss der Schlüssel noch um den Mandanten und das WJ erweitert werden, damit der Datensatz eindeutig ist. Wird ExpandKey nicht angegeben, wird der Wert anhand des Typs der Tabelle automatisch gesetzt.

Rückgabe: Wenn der Datensatz gefunden wurde, wird TRUE zurückgegeben, andernfalls FALSE.

BOOL Update ()

Mit dieser Methode kann der zuletzt geladene Datensatz aktualisiert werden.

Wenn die Tabellenvariablen gültige Werte enthalten, muss der Datensatz zuvor nicht mit Get geladen werden.

Rückgabe: TRUE, wenn der Datensatz aktualisiert werden konnte

BOOL Insert ()

Mit dieser Methode kann ein neuer Datensatz eingefügt werden. Die Tabellenvariablen müssen zuvor mit den neuen Werten versorgt werden.

Rückgabe: TRUE, wenn der Datensatz eingefügt werden konnte

BOOL Delete (BSTR Key, VARIANT WhereStmt)

Mit dieser Methode kann einer oder mehrere Datensätze gelöscht werden.

Parameter

Key	Der eindeutige Schlüsselwert, der in der beim OpenTable angegebenen Spalte drinsteht
WhereStmt	Wird ein WhereStmt übergeben, dann werden alle Datensätze gelöscht, die dem WhereStmt entsprechen. Im Key muss dann ein Leerstring übergeben werden.

Rückgabe: Wenn kein Fehler auftrat, wird TRUE zurückgegeben, andernfalls FALSE.

void CopyToWindow (short Window)

Mit dieser Methode können die Tabellenvariablen in die Variablen der View 495 kopiert werden (die in MDP Projekten als benutzerdefinierte Variablen verwendet werden). Mit dem Aufruf werden bestehende Variablen überschrieben, bzw. verändert so dass sie den Typen der Spaltenwerte der Tabelle entsprechen.

Parameter

Window	Das Fenster, in dem die Variablen verändert werden sollen.
--------	--

CWLSearchResult *Select (BSTR SelectStmt)

Mit dieser Methode kann eine SELECT - Abfrage auf die Tabelle durchgeführt werden. Die Spaltenwerte der gefundenen Datensätze werden in die Tabellenvariablen der Tabelle kopiert. Die Werte können auch mit Methoden des CWLSearchResult - Objekts ausgelesen werden.

Die Funktion ersetzt im SelectStmt ~~~~ durch den aktuellen Mandanten (Textspalte) und yyyy durch das aktuelle Wirtschaftsjahr (numerisch).

Parameter

SelectStmt	Der WHERE - Ausdruck, der in einem SELECT * auf die Tabelle verwendet wird.
------------	---

Rückgabe: ein CWLSearchResult Objekt

Beispiel

```
Set tDW = conn.OpenTable2 (699, 900) ' benutzerspez. Tabelle öffnen
' alle Datensätze sortiert nach der spalte U000
Dim search
Set search = tDW.Select("order by U000")
If search.RowCount > 0 Then
    Do
        ' den AN aus der tDW laden
        T401.get tDW.value(0)
        grid.AddLine
        If search.NextRecord = False Then ' der letzte Datensatz?
            Exit Do
        End If
    Loop
End If
```

6. Klassen

Folgende Klassen bestehen im CWL Objektmodell, aus denen Objekte vom Root Objekt **CWLStart** abgeleitet werden.

6.1. CWLCompany

Dieses Objekt dient als Stellvertreter des aktuell geladenen Mandanten. Es können die aktuellen Mandantendaten ausgelesen werden. Zusätzlich können beliebige SQL Select Abfragen ausgeführt werden um jede vorhandene Tabelle des Mandanten auslesen zu können.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLCompany

Eigenschaften

BSTR **Nr**

BSTR **Name**

ICWLSearchResult ***SearchResult**

BOOL **Valid**

VARIANT **Value** (short nVar)

CWLDbConnection ***Connection**

Int **CompanyYear**

ICWLWindowVars* ModifiedVars

VARIANT **Property** (BSTR ObjectKey, int PropertyGroup, int PropertyType, BSTR CompanyForCRM)

Methoden

ICWLSearchResult ***SearchRecord** (BSTR strTableName, BSTR strWhereStatement)

long **UpdateRecord** (BSTR strTableName, BSTR strUpdateStatement, BSTR strWhereStatement)

void **Refresh** ()

CWLDbConnection **GetSystemConnection** (CWLSystemServerType what)

BSTR **ConvertCompanyYearToString** (int YearValue)

int **ConvertCompanyYearStringToValue** (BSTR YearString)

Events

OnUpdateTable (short TableNum)

OnInsertTable (short TableNum)

OnDeleteTable (short TableNum, BSTR Key, BSTR WhereStmnt)

6.1.1. Eigenschaften

Nr [BSTR, read only]

Liefert die Mandantennummer.

Name [BSTR, read only]

Liefert den Mandantenamen.

WinLine Objektmodelle

mesonic © 02/2023

Valid [BOOL, read only]

Gibt true zurück, wenn der aktuelle Mandant geladen ist.

Value (short nVar) [VARIANT, read only]

Liefert den Wert der Spalte **varNo** des aktuell geladenen Mandanten. Spalten die nicht vorhanden sind werden mit dem Variablentyp "Nothing" zurückgegeben (Im Mandanten sind nicht alle Spalten durchgängig numeriert). Wird eine ungültige Variablen Nummer übergeben erfolgt eine Runtime Error.

SearchResult [ICWLSearchResult *, read only]

Liefert das Ergebnis Objekt, das die Ergebniswerte enthält (siehe CWLSearchResult - Objekt).

Connection [CWLDbConnection *, read only]

Liefert als Ergebnis das CWLDbConnection Objekt des aktuellen Mandanten.

CompanyYear [int, read only]

Liefert als Ergebnis das aktuelle Wirtschaftsjahr im internen numerischen Format, das in der Wirtschaftsjahr Combobox eingestellt ist. Der Wert kann mit der Funktion **ConvertCompanyYearStringToValue** in das Textformat umgewandelt werden.

ModifiedVars [ICWLWindowVars*, read only]

Liefert ein CWLWindowVars - Objekt, das die Daten beinhaltet, die im Änderungs-Event verändert werden (OnUpdateTable, OnInsertTable, OnDeleteTable).

Beispiel:

```
Sub CWLCompany_OnUpdateTable(TableNo)
    If TableNo = 51 Then
        Set myvars = CWLCompany.ModifiedVars
        general.msgbox "OnUpdateTable Event. Feld 'zu Handen' ist: " & myvars.value(51,53)
    End If
End Sub
```

Property (BSTR ObjectKey, int PropertyGroup, int PropertyNumber, BSTR CompanyNumberCRM) [VARIANT, read only]

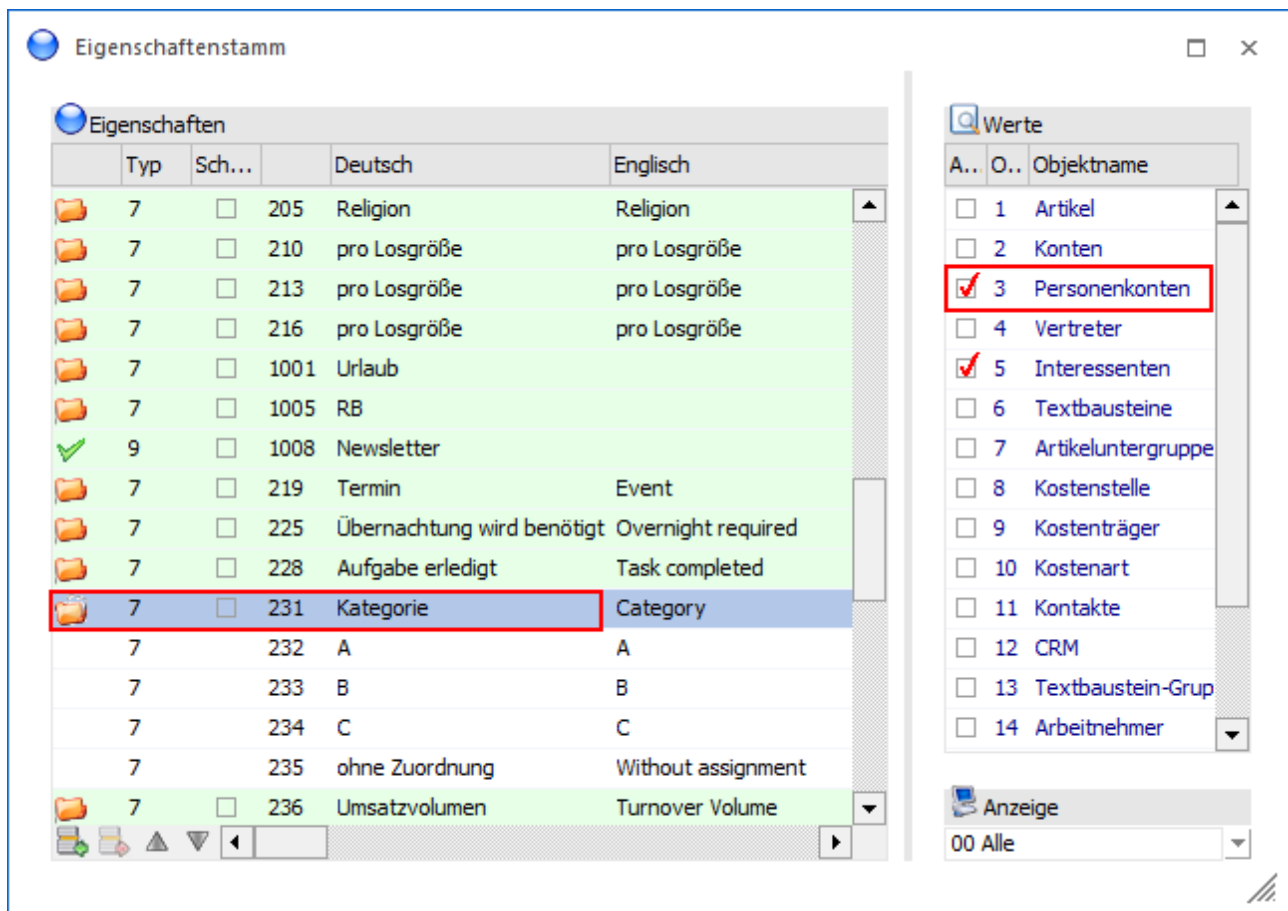
Liefert den Wert der Eigenschaft **PropertyNumber** (in der Eigenschaften Gruppe **PropertyGroup**) für das Objekt **ObjectKey** (das kann z.b. eine Kontonummer sein). Falles es sich um eine CRM - Eigenschaft handelt, und der aktuelle Mandant ist nicht der Hauptmandant für ein mandantenübergreifendes CRM, dann muss für **CompanyNumberCRM** jener Hauptmandant angegeben werden, andernfalls muss ein leerer Text übergeben werden.

Ist die Eigenschaft nicht vorhanden, oder das gewünschte Objekt enthält nicht diese Eigenschaft, wird ein leerer Text zurückgegeben.

Beispiel:

```
Status = CWLStart.CurrentCompany.Property ("230A001", 3, 231, "")
```

Damit wird die Eigenschaft 231, die dem Eigenschaftentyp 3 unterliegt, für das Konto 230A001 ausgelesen. Zur Verdeutlichung des Verweises auf die Eigenschaft:



6.1.2. Methoden

Refresh

Es werden die im Programm hinterlegten Mandantendaten aus der Datenbank erneut eingelesen.

SearchRecord (BSTR strTableName, BSTR strWhereStatement)

In der Tabelle **strTableName** wird nach dem Datensatz entsprechend des Parameters **strWhereStatement** gesucht. Wird der Datensatz gefunden, können die einzelnen Ergebniswerte mit dem zurückgegebenen CWLSearchResult Objekt ausgelesen werden oder es wird die Eigenschaft **SearchResult** ausgewertet.

Anstelle des Mandanten in der Abfrage kann ~~~~ übergeben werden, dann wird der aktuell geladene Mandant an dieser Stelle eingesetzt (z.B. wird aus WHERE MESOCOMP = '~~~~' WHERE MESOCOMP = '300M'), gleiches gilt für das aktuelle Wirtschaftsjahr, das durch yyyy eingesetzt werden kann (z.B. WHERE MESOYEAR = yyyy).

strWhereStatement enthält die Bedingung welcher Datensatz gesucht werden soll (z.B. 'C002 = '10001'). Ab der Version 8.0 muss in dieser Bedingung auch der aktuelle Mandant und das aktuelle Wirtschaftsjahr abgefragt werden!

Es wird immer nur der erste gefundene Datensatz in **SearchResult** zur Verfügung gestellt - wenn sich das Ergebnis aus mehreren Datensätzen zusammensetzt, gehen die Ergebnisse der anderen Datensätze verloren.

Ist die Abfrage syntaktisch nicht in Ordnung oder wird versucht eine nicht existierende Tabelle anzusprechen erfolgt ein Runtime Error.

Wird der Datensatz nicht gefunden enthält SearchResult als MaxColumnIndex -1. MaxColumnIndex ist gleichzeitig die Standardeigenschaft von SearchResult, weswegen die Abfrage auch folgendermaßen ausgeführt werden kann:

```
Set result = CWLStart.CurrentCompany.SearchRecord (...)
if result = -1 then ' result ist gleichwertig zu result.MaxColumnIndex
    MsgBox "Datensatz nicht gefunden"
end if
```

Parameter

strTableName	Name der Tabelle
strWhereStatement	Bedingung für die Abfrage (gültiger SQL Ausdruck, wie er im WHERE - Teil des SQL Ausdrucks vorkommen darf) Es muss immer auch auf den Mandanten und das Wirtschaftsjahr eingeschränkt werden (MESOCOMP = '~~~~' and MESOYEAR = yyyy)

Rückgabewert (ICWLSearchResult *)

UpdateRecord (BSTR strTableName, BSTR strUpdateStatement, BSTR strWhereStatement)

In der Tabelle **strTableName** werden die Spalten mit den Werten, die in **strUpdateStatement** übergeben werden, für alle Datensätze entsprechend dem **strWhereStatement** upgedatet. (z.B. UPDATE strTableName SET strUpdateStatement WHERE strWhereStatement → UPDATE T024 SET C003 = 'Herren Rennsportrad' WHERE C002 = '10005' and MESOCOMP = '~~~~' and MESOYEAR = yyyy).

Können keine Werte upgedatet werden (das **strWhereStatement** entspricht keinem existierenden Datensatz) wird FALSE zurückgegeben.

Ist die Abfrage syntaktisch nicht in Ordnung oder wird versucht eine nicht existierende Tabelle anzusprechen erfolgt ein Runtime Error.

Parameter

strTableName	Name der Tabelle
strUpdateStatement	Angabe der Werte für die Spalten, die aktualisiert werden sollen (z.B. "C001 = 3, C002 = 'Text'")
strWhereStatement	Bedingung für die Abfrage (gültiger SQL Ausdruck, wie er im WHERE - Teil des SQL Ausdrucks vorkommen darf) Es muss immer auch auf den Mandanten und das Wirtschaftsjahr eingeschränkt werden (MESOCOMP = '~~~~' and MESOYEAR = yyyy)

Rückgabewert (long)

> 0	Anzahl der erfolgreich aktualisierten Datensätz.
0	Das Update wurde nicht durchgeführt.

CWLDbConnection GetSystemConnection (CWLSYSTEMSERVERTYPE what)

Gibt das CWLDbConnectionobjekt für die gewünschte Systemdatenbank zurück (vgl. Die CWLSYSTEMSERVERTYPE Konstanten im Anhang).

BSTR ConvertCompanyYearToString (int YearValue)

Konvertiert das Wirtschaftsjahr vom internen numerischen Format in das Textformat, wie es in der Wirtschaftsjahr – Combobox dargestellt wird.

Int ConvertCompanyYearStringToValue (BSTR YearString)

Konvertiert das Wirtschaftsjahr vom Textformat wie es in der Wirtschaftsjahr – Combobox dargestellt wird, in das interne numerische Format.

6.1.3. Events

OnUpdateTable (short TableNum)

Dieses Event wird ausgelöst, wenn ein Update auf eine mesonic-Datenbanktabelle erfolgt, die mit benutzerdefinierten Spalten erweitert wurde.

Der Programmierer kann hier vor dem tatsächlichen Update in der Datenbank die zu schreibenden Daten bearbeiten.

Auf die Daten, die nun modifiziert werden, kann mit der Eigenschaft CWLCompany.ModifiedVars zugegriffen werden.

Parameter

TableNum	Tabelle, die mit Benutzerspalten erweitert wurde
----------	--

OnInsertTable (short TableNum)

Dieses Event wird ausgelöst, wenn ein Insert auf eine mesonic-Datenbanktabelle erfolgt, die mit benutzerdefinierten Spalten erweitert wurde.

Der Programmierer kann hier vor dem tatsächlichen Insert in die Datenbank die zu schreibenden Daten bearbeiten.

Auf die Daten, die nun eingefügt werden, kann mit der Eigenschaft CWLCompany.ModifiedVars zugegriffen werden.

Parameter

TableNum	Tabelle, die mit Benutzerspalten erweitert wurde
----------	--

OnDeleteTable (short TableNum, BSTR Key, BSTR WhereStmt)

Dieses Event wird ausgelöst, wenn ein Delete auf eine mesonic-Datenbanktabelle erfolgt, die mit benutzerdefinierten Spalten erweitert wurde.

Der Programmierer kann abhängig von den zu löschenden Datensätzen eigene Operationen anstoßen. Auf die Daten, die nun gelöscht werden, kann mit der Eigenschaft CWLCompany.ModifiedVars zugegriffen werden. Parameter

TableNum	Tabelle, die mit Benutzerspalten erweitert wurde
Key	Die Keyspalte des zu löschenden Datensatzes. Wenn der Wert leer ist, wird das <i>WhereStmt</i> für das Löschen verwendet.
WhereStmt	Eine 'where'-Abfrage, die die zu löschenden Datensätze auswählt. Der Ausdruck wird nur verwendet, wenn <i>Key</i> leer ist. Sind beide leer, werden alle Datensätze des aktuellen Mandanten gelöscht.

6.2. CWLDbConnection

Dieses Objekt beschreibt eine Datenbankverbindung. Neben dem Typ der Datenbank wird der Name der Datenbank und der Name des Servers beschrieben.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLDbConnection**Eigenschaften**

CWLDbConnectionType **Type**

BSTR **DatabaseName**

BSTR **ServerName**

Methoden

CWLSearchResult ***Select** (BSTR Statement);

CWLTable* **OpenTable** (BSTR strTableName, int ViewNumber, BSTR KeyColumn, int WindowId, VARIANT UseCompany);

void **CloseTable** (CWLTable* pTable);

BOOL **ExecuteSQL** (BSTR Statement);

CWLTable* **OpenTable2** (short Number, short WindowId, VARIANT KeyColumn);

6.2.1. Eigenschaften**Type [CWLDbConnectionType, read only]**

Liefert den Typ der Datenbank (vgl. die CWLDbConnectionType - Konstanten im Anhang).

DatabaseName [BSTR, read only]

Liefert den Namen der Datenbank.

ServerName [BSTR, read only]

Liefert den Namen des Servers.

6.2.2. Methoden

Select (BSTR Statement)

Es wird eine SQL-Anweisung auf der Datenbankverbindung ausgeführt. Die Anweisung muss eine SELECT-Anweisung sein, die das (NOLOCK) - Attribut enthalten muss.

Parameter

Statement	Der SQL Ausdruck (muss eine SELECT-Anweisung sein)
-----------	--

Rückgabewert: ein CWLSearchResult - Objekt mit den Ergebnissen der Abfrage

Beispiel

```
Dim conn, result
'Datenbankverbindung des aktuellen Mandanten
Set conn = CWLStart.CurrentCompany.Connection

' den aktuellen Mandanten (mit akutellem WJ) auslesen
Set result = conn.Select ("Select * from t001 (NOLOCK) where mesocomp = '~~~~' And
mesoyear = yyyy")

' den Namen des aktuellen Mandanten ausgeben
general.MsgBox result.value("c000")
```

CWLTable OpenTable (BSTR strTableName, int ViewNumber, BSTR KeyColumn, int WindowId, VARIANT UseCompany)

Die Methode öffnet die Tabelle mit dem angegebenen Namen innerhalb der aktuellen Datenbankverbindung. Diese Funktion wird bei Tabellen verwendet, die nicht das Namensschema der mesonic-Tabellen verwenden (Txxx).

Parameter

strTableName	Der Name der Tabelle
ViewNumber	Die Nummer der Tabelle, mit der die Spaltenwerte als Variable angelegt werden. Die Variablen werden in dem mit WindowId angegebenen Fenster angelegt und dürfen von keiner anderen geöffneten Tabelle verwendet werden.
KeyColumn	Die Spalte in der Tabelle, die in den CWLTable - Methoden verwendet wird, die einen Schlüssel als Parameter verwenden (z.B. get)
WindowId	Das Programmfenster, in dessen

	Variablen die Variablen für die Spalten der Tabelle angelegt werden (können dann mit CwlWindow.Vars(ViewNumber,x) verwendet werden)
UseCompany	Falls mit dieser Methode eine Standardtabelle geöffnet wird, kann mit diesem optionalen Parameter übergeben werden, ob der verwendete Schlüssel bei einem get/delete automatisch erweitert werden soll (wird eine Txxx Tabelle verwendet, dann setzt das Programm den Parameter automatisch auf true. Wenn eine Keyspalte übergeben wurde, die nicht dem Defaultschlüssel der Tabelle entspricht, muss der Parameter mit false übergeben werden, weil sonst ein falscher Key erzeugt wird.

Rückgabewert: ein CWLTable - Objekt

Beispiel

```
On Error Resume Next
Dim conn, table
'Datenbankverbindung des aktuellen Mandanten
Set table = conn.OpenTable ("MeineTabelle", 497, "Nr", 900)
If table Is Nothing Then
    MsgBox "Die Tabelle 'MeineTabelle' wurde nicht gefunden"
End If
If Not table.get ("1") Then
    MsgBox "Get von 'MeineTabelle': der Datensatz wurde nicht gefunden!"
Else
    MsgBox "Get von 'MeineTabelle (1)': " & table.value(1)
End If
```

CloseTable (CWLTable Table)

Schließt die geöffnete Tabelle und gibt die angelegten Variablen wieder frei.

Parameter

Table	Das CWLTable Objekt, das mit OpenTable geöffnet wurde
-------	---

BOOL ExecuteSQL (BSTR Statement)

Die Methode führt einen beliebigen SQL-Ausdruck aus. Select-Ausdrücke werden zwar ausgeführt, dafür ist die Funktion aber nicht gedacht, da die Ergebnisse des Selects nicht ausgelesen werden.

Parameter

Statement	Der SQL Ausdruck
-----------	------------------

Rückgabe: Bei einem Fehler wird FALSE zurückgegeben, andernfalls TRUE.

Hinweis:

Es kann eine Stored Procedure am SQL-Server mit dieser Methode in einem CTK-Fensterskript ausgeführt werden. Wenn mehrere SQL-Ausdrücke in der Stored Procedure enthalten sind, kann es hierbei von Vorteil

sein, das "SET NOCOUNT ON" Eigenschaft in der Stored Procedure zu setzen. Wenn die Eigenschaft nicht gesetzt ist, wird eine Rückgabe für jeden SQL-Ausdruck in einer Stored Procedure getätigt. Die WinLine wertet allerdings in diesem Fall nur die Rückgabe vom ersten SQL-Ausdruck aus im CTK-Script und die Methode wird danach abgeschlossen. Zusätzlich bewirkt die gesetzte Eigenschaft, dass die Stored Procedure an sich schneller ausgeführt werden kann.

CWLTable OpenTable2 (short Number, short WindowId, VARIANT KeyColumn)

Die Methode öffnet die Tabelle mit der Nummer 'Number', wobei die Tabelle einen der mesonic Konvention folgenden Tabellennamen besitzen muss (Txxx, wobei xxx eine Zahl mit führenden Nullen ist).

Diese Methode entspricht der OpentTable - Methode, kann aber nur bei Tabellen mit dem Namen Txxx verwendet werden.

Parameter

Number	Die Tabellennummer (xxx im Tabellennamen Txxx)
WindowId	Das Fenster, in dem die Variablen für die Tabellenspalten angelegt werden.
KeyColumn	Die Spalte, die bei nachfolgenden get, update oder delete Aufrufen als Schlüsselspalte dient. Wird der Parameter nicht angegeben, wird die Default-Schlüsselspalte verwendet.

Rückgabe: ein CWLTable Objekt

6.3. CWLModule

Objekte von dieser Klasse repräsentieren ein CWL Modul wie START, FIBU, FAKT, etc. - siehe dazu auch Konstanten-CWLApplicationNr.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLModule

Eigenschaften

ICWLWindow* **CurrentWindow**

ICWLWinCollection* **Windows**

BSTR **Name**

short **Number**

Methoden

BOOL **Activate** ()

BOOL **IsWindowOpen** (short WinId)

int **SendWindowEvent** (short WinId, int EventType, VARIANT Data[, BOOL bPostMessage])

6.3.1. Eigenschaften

CurrentWindow [ICwlWindow*, read only]

Liefert einen Pointer auf das aktuelle Fenster in diesem Modul. Wenn kein Fenster geöffnet ist, wird nichts zurückgegeben. Ein Runtime Error erfolgt, wenn es innerhalb der Funktion zu einem unerwarteten Problem kommt.

Ein Script Fenster des Typs **cwlScriptWindowStandard** wird ebenfalls als aktuelles Fenster zurückgegeben. Alle anderen Script Fenster Typen fallen nicht in diese Kategorie und werden nicht als CurrentWindow erkannt.

Beispiel 1 (prüft auf Runtime Error bei fehlendem Fenster)

On Error Resume Next

err.clear

myname = cwlstart.CurrentModule.CurrentWindow.Name

If err <> 0 Then

myname = "Kein Fenster Aktiv:" & err.number

End If

On Error Goto 0

MsgBox myname

Beispiel 2 (prüft auf nicht existierendes Fenster)

Set curwin = CurrentModule.CurrentWindow

If TypeName(curwin) = "Nothing" Then

msgbox "Kein Fenster Aktiv"

Else

msgbox curwin.Name

End If

Windows [ICwlWinCollection*, read only]

Liefert einen Pointer auf ein Objekt der Klasse **CWLWinCollection**, mit dem auf alle aktiven Fenster eines Moduls zugegriffen werden kann.

Fenster müssen zwar nicht sichtbar, aber geladen sein, damit auf sie über diese Collection zugegriffen werden kann.

Name [BSTR, read only]

Name des Moduls.

Number [short, read only]

Nummer des Moduls.

Die aktuell vergebenen Nummern sind die unter **CWLApplicationNr** angeführten Nummern.

6.3.2. Methoden

Activate

Aktiviert das entsprechende Modul. Das Modul muß bereits einmal aktiviert worden sein (d.h. es muß schon 'vorhanden' sein). Im Gegensatz zu **MacroCommands. MApplication**(ApplicationNr) wird das Modul nicht gestartet, sondern nur auf ein bereits gestartetes Modul umgeschaltet.

Rückgabewert (VARIANT_BOOL)

TRUE	Modul konnte gestartet werden
FALSE	Modul konnte nicht gestartet werden (z.B. fehlende Berechtigung)

Beispiel

'wenn das Modul noch nicht aktiviert wurde, kann auf das Modul nicht zugegriffen werden

```
if TypeName (cwlstart.Module(cwlFAKT)) = "Nothing" then
    exit sub
endif
myresult = cwlstart.Module(cwlFAKT).activate
```

BOOL IsWindowOpen (short WinId)

Diese Methode prüft, ob das Fenster mit der angegebenen Nummer geöffnet ist.

Parameter

WinId	Fenster Nr des gewünschten Fensters
-------	-------------------------------------

Rückgabe: TRUE wenn das Fenster geöffnet ist, sonst FALSE

int SendWindowEvent (short WinId, int EventType, VARIANT Data[, BOOL bPostMessage])

Diese Methode sendet an das angegebene Fenster ein Event mit den Daten *EventType* und *Data*. In dem Zielfenster kann das Event mit dem CWLWindow-Event *OnUserEvent* abgefragt werden.

Die Methode gibt den Wert zurück, der im Event *OnUserEvent* gesetzt wird. Wird dort kein Wert gesetzt, oder es wird das Event mit *bPostMessage* = TRUE gesendet, ist der Rückgabewert 0.

Mit dem optionalen Parameter *bPostMessage* kann erreicht werden, dass das Event erst ausgelöst wird, nachdem die Methode bereits zurückgekehrt ist. Dies kann z.B. notwendig sein, wenn das Fenster, das diese Methode aufruft, innerhalb des Events geschlossen wird. Dies würde dazu führen, dass der Aufruf zurückkehrt und das aufrufende Script nicht mehr da ist, was direkt zu einem Programmabsturz führen würde. Mit gesetztem *bPostMessage* wird das Event erst ausgeführt, wenn das Script mit der Abarbeitung der Methode fertig ist.

Ein sinnvoller Anwendungsfall wäre die Kommunikation zwischen zwei Fenstern, die bisher nur mit dem globalen Objekt möglich war.

Parameter

WinId	Fenster Nr des gewünschten Fensters
EventType	beliebig wählbare Zahl, um das Event zu kennzeichnen. Wird im OnUserEvent als Parameter übergeben.
Data	zusätzliche Daten (dies können beliebige Werte, bzw. auch Arrays sein). Der Wert wird im OnUserEvent als Parameter übergeben
bPostMessage (optional)	Das Event erst auslösen, wenn die Methode bereits zurückkehrt ist. Der Parameter ist optional und kann weggelassen werden. Der Default ist FALSE.

Rückgabe: Wert, der im OnUserEvent als Ergebnis gesetzt wird. Wird bPostMessage = TRUE verwendet, dann ist der Wert immer 0.

6.3.3. Verwendung

Einen Pointer auf ein Objekt der Klasse **CWLModule** kann man auf folgende Weisen erhalten:

Aktuelles Modul

```
myModule = CWLStart.CurrentModule
```

Beliebiges Modul

```
myModule = CWLStart.Module(cwIFAKT)
```

6.4. CWLWinCollection

Bietet Zugriff auf alle Objekte der Klasse **CWLWindow**.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLWinCollection

Eigenschaften

long **Count**

ICwlWindow* **Item** (long nWinId)

ICwlWindow* **NamedItem** (BSTR strWinName)

ICwlWindow* **IndexedItem** (int nIndex)

Methoden

BOOL **Add** (long nWinId)

6.4.1. Eigenschaften

Count [long, read only]

Anzahl der Objekte in dieser Collection.

6.4.2. Methoden

Item (long nWinId)

Liefert einen Pointer auf das Fenster mit der angegebenen ID **nWinId**.

Die ID entspricht jener im CTK.

Existiert das Fenster mit der angegebenen ID nicht, enthält der Rückgabewert nothing (muß mit Typename (var) = "Nothing" geprüft werden).

Parameter

nWinId	Fenster Nr des gewünschten Fensters
--------	-------------------------------------

Rückgabewert (ICwlWindow*)
 Pointer auf das Fenster

NamedItem(BSTR strWinName)

Liefert einen Pointer auf das Fenster mit dem angegebenen Namen **strWinName**.

Der Name entspricht der Eigenschaft Title des Fensters im CTK.

Existiert das Fenster mit der angegebenen ID nicht, enthält der Rückgabewert nothing (muß mit Typename (var) = "Nothing" geprüft werden).

Parameter

strWinName	Der Fenster Titel des gewünschten Fensters
------------	--

Rückgabewert (ICwlWindow*)
 Pointer auf das Fenster

IndexedItem(int nIndex)

Liefert einen Pointer auf das Fenster mit dem Index **nIndex** in der Collection, beginnend bei 0.

Existiert das Fenster mit der angegebenen ID nicht, enthält der Rückgabewert nothing (muß mit Typename (var) = "Nothing" geprüft werden).

Parameter

nIndex	Der Index des gewünschten Fensters (Index aller offenen Fenster, beginnend mit 0)
--------	---

Rückgabewert (ICwlWindow*)
 Pointer auf das Fenster

Add(long nWinId)

Öffnet das Fenster mit der Fenster Id **nWinId** in diesem Modul. Der Befehl entspricht dem Makrobefehl **MacroCommands.MWindow**.

Kann das Fenster nicht geöffnet werden, wird FALSE zurückgegeben.

Parameter

nWinId	Die Nummer des gewünschten Fensters
--------	-------------------------------------

6.4.3. Verwendung

Ein Objekt der Klasse **CWLWinCollection** existiert nur in der **Windows** Property eines Moduls (**CWLModule**).

Diese Collection enthält immer alle geladenen Fenster (welche sichtbar oder unsichtbar sein können) und dient zum Zugriff auf diese.

z.B. Zugriff auf das **Artikel - Stammdaten** Fenster:

```
myWindow = CWLStart.Module(cwlFAKT).Windows.Item(210)
```

6.5. CwlWindow

Diese Klasse definiert Fenster Objekte in der Collection **CWLWinCollection**.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLWindow
Eigenschaften
short CurrentField
BOOL Visible
short Id
ICwlWindowVars* Vars
BSTR Name
CWLWindowTypes Type
ICwlFgCollection* Controls
ICwlFgControl* CurrentControl
BSTR CurrentFilter
Int CurrentCompanyYear
Methoden
long Close ()
void Activate ()
void Refresh ()
CwlReport CreateReport (short Type, BSTR Name, VARIANT left, VARIANT top, VARIANT width, VARIANT height, VARIANT Description, VARIANT SpoolfileName)
void CloseReport (CwlReport Report)
void SetShowLevel (short VonLevel, short BisLevel, VARIANT setzen)
void CallWindowOnClose (short AppId, short WindowId)

6.5.1. Eigenschaften

CurrentField [short, read write]

ID des Feldes (Element) welches in diesem Fenster den Focus hat.

Das Versetzen des Focus simuliert das Verlassen des aktuellen Feldes und überprüft, ob das Feld verlassen werden konnte. Erst wenn dies funktioniert (d.h. die Applikationslogik es zuläßt) wird der Focus auf das gewünschte Feld gesetzt.

Wird die Id 0 verwendet so wird dadurch der Focus auf das nächste Feld in der TAB Reihenfolge gesetzt.

Kann der Focus nicht versetzt werden, wird ein Runtime Error ausgelöst.

Visible [BOOL, read write]

Bestimmt, ob ein Fenster sichtbar ist oder nicht.

Diese Eigenschaft kann für UserForms auch gesetzt werden, für CWL Systemfenster nur gelesen werden.

Id [short, read only]

Eindeutige ID des Fensters. Entspricht der ID im CTK.

Die ID von UserForms und Preview Fenstern wird dynamisch vergeben und hängt von der Anzahl der geöffneten Fenster dieses Typs ab.

Vars [ICwlWindowVars*, read only]

Erlaubt Zugriff auf die in diesem Fenster verwendeten Variablen.

Name [BSTR, read only]

Name des Fensters.

Type [CWLWindowTypes, read only]

Typ des Fensters. Siehe auch Konstanten - CWLWindowTypes.

Controls [ICwlFgCollection*, read only]

Enthält eine Collection mit allen Elementen des Fensters der Klasse **CWLfgControl**.

CurrentControl [ICwlFgControl*, read only]

Zeiger auf das Feld (Element) welches in diesem Fenster den Focus hat.

CurrentFilter [BSTR, read only]

Der Name des aktuell eingestellten Filters (nur wenn das Fenster eine Filter-Combobox im Toolbar aufweist).

CurrentCompanyYear [int, read only]

Das aktuelle Wirtschaftsjahr im internen numerischen Format. Das Format kann mit Funktionen aus dem CWLCompany Objekt zwischen dem Text- und dem numerischen Format konvertiert werden (ConvertCompanyYearToString und ConvertCompanyYearStringToValue).

6.5.2. Methoden

Close

Schließt das Fenster.

Kann nur bei CWL Standardfenstern verwendet werden und simuliert das Aktivieren des ENDE Buttons im Fenster.

Wird versucht ein Fenster eines anderen Typs zu schließen, wird von der Funktion 0 zurückgegeben. War das Schließen des Fensters erfolgreich wird 1 zurückgegeben.

Refresh

Alle Felder im Fenster werden neu dargestellt. Dadurch werden Änderungen an Variablen, die durch die Elemente dargestellt werden, sichtbar gemacht.

CwlReport CreateReport (short Type, BSTR Name, VARIANT left, VARIANT top, VARIANT width, VARIANT height, VARIANT Description, VARIANT SpoolfileName)

Die Funktion erzeugt ein CwlReport - Objekt, mit dem eine Auswertung ausgegeben werden kann.

Ausgehend von einer Reportdefinition im CWLPDFE.EXE, wird die Auswertung entweder am Bildschirm oder direkt am Drucker (mit dem Parameter *Type*) ausgegeben.

Wird in den Parametern *left*, *top*, *width* oder *height* ein Wert übergeben (nicht 0), dann werden bei der Ausgabe am Bildschirm immer diese Werte verwendet, egal ob die Ansicht gespeichert wurde oder nicht. Bleiben die Werte auf 0, dann wird zu Beginn die Standardposition und -größe einer Bildschirmauswertung verwendet, die dann aber mit der rechten Maustaste gespeichert werden kann, und beim erneuten Aufruf automatisch verwendet wird.

Parameter

Type	Ziel der Auswertung: 1... Ausgabe am Bildschirm 2... Ausgabe am Drucker 4... Ausgabe in den Spooler
Name	Name des Reports, wie er im CWLPDFE erzeugt wurde
left	Linke Position der Auswertung am Bildschirm (in Basiseinheiten)
top	Obere Position der Auswertung am Bildschirm (in Basiseinheiten)
width	Breite der Auswertung am Bildschirm (in Basiseinheiten)
height	Höhe der Auswertung am Bildschirm (in Basiseinheiten)
Description	Beschreibung der Auswertung (max 50 Zeichen), die in der Spooldatei gespeichert wird
SpoolfileName	Der Name der Spooldatei, wenn in eine Spooldatei ausgegeben wird. Wird der Parameter nicht angegeben, wird in die Spooldatei des Benutzers gedruckt, die im Fenster "Despool" bearbeitet werden kann.

Rückgabewert (CwlReport)

Der erzeugte Report.

void CloseReport (CwlReport Report)

Diese Funktion schließt einen Report. Danach kann der Report nicht mehr für weitere Ausgaben verwendet werden.

Die Funktion kann z.B. als Reaktion auf das Cancel - Event des Reports aufgerufen werden, um die Auswertung am Bildschirm zu schließen (das Event wird beim Klick auf den Stop-Button der Auswertung, oder beim Schließen des Auswertefensters aufgerufen).

Parameter

Report	Der CwlReport, der mit CreateReport angelegt wurde.
--------	---

void SetShowLevel (short VonLevel, short BisLevel, VARIANT setzen)

Die Funktion setzt oder entfernt die angegebenen ShowLevel. Unter ShowLevel versteht das Programm Darstellungsebenen, die ein- und ausgeschaltet werden können. Wird ein ShowLevel gesetzt, werden alle Elemente, die auf dieser Ebene liegen angezeigt. Alle auf der Ebene liegenden Elemente werden damit automatisch aktiv. Deshalb müssen nach dem Umschalten eines ShowLevels zuvor ausgegraute Elemente erneut deaktiviert werden (fg.Active = false).

Parameter

VonLevel	ab diesem ShowLevel
BisLevel	Bis zu diesem Showlevel
setzen	TRUE: die ShowLevel werden angezeigt (der Defaultwert, wenn der Parameter weggelassen wird) FALSE: die Showlevel werden

	versteckt
--	-----------

void CallWindowOnClose (short AppId, short WindowId)

Die Funktion hinterlegt im Fenster einen Fensteraufruf, der beim Schließen des aktuellen Fensters ausgeführt wird.

Parameter

AppId	Applikationsnummer des aufzurufenden Fensters
WindowId	Die Fensternummer, die nach dem Schließen des aktuellen Fensters aufgerufen wird

6.6. CwIFgCollection

Bietet Zugriff auf eine Collection von Objekten der Klasse **CWLFgControl**.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CwIFgCollection

Eigenschaften

long **Count**
 ICwIFgControl* **Item** (long nFgId)
 ICwIFgControl* **IndexedItem** (long nIndex)

6.6.1. Eigenschaften

Count [long, read only]

Anzahl der Objekte in dieser Collection.

6.6.2. Methoden

Item(long nFgId)

Liefert ein Objekt der Klasse **CWLFgControl** aus dieser Collection. Mit **nFgId** kann die ID des Elements angegeben werden. Diese ID entspricht jener im CTK.

Parameter

nFgId	Die Nummer des gewünschten Fg Controls
-------	--

Rückgabewert (ICwIFgControl*)
 Pointer auf das Element

IndexedItem(long nIndex)

Liefert ein Objekt der Klasse **CWLFgControl** aus dieser Collection. **nIndex** entspricht der Position des Elements in der Collection (0..**Count**).

Parameter

nIndex	Der Index des gewünschten Fg Controls in der Liste der existierenden Controls, beginnend mit 0
--------	--

Rückgabewert (ICWLFgControl*)
 Pointer auf das Element

6.6.3. Verwendung

Jedes Objekt der Klasse **CWLWindow** enthält eine Eigenschaft **Controls** der Klasse **CWLFgCollection**. Damit kann man auf alle Elemente eines Fensters (Eingabefelder, Texte, etc.) zugreifen, die ID der Elemente erhält man aus dem CTK.

Beispiel für den Zugriff auf das Feld **Artikelnummer** (ID=101) im Fenster **Artikel - Stammdaten** (ID=245) im Modul **FAKT** (ID=cwIFAKT):

`Set myElement = CWLStart.Module(cwIFAKT).windows.item(245).controls.item(101)`

6.7. CwLFgControl

Ein Objekt dieser Klasse repräsentiert ein Element in einem Fenster. Dabei kann es sich um eine Inputbox, um ein Label oder jedes andere Element in einem Fenster handeln. Siehe dazu auch Konstanten - CWLControlTypes.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLFgControl**Eigenschaften**

short **Id**
 VARIANT **Contents**
 BSTR **Text**
 long **View**
 long **Var**
 long **Line**
 long **Column**
 CWLControlTypes **Type**
 long **Font**
 long **Height**
 long **Width**
 ICwIPreview* **Preview**
 ICwISpreadSheet* **SpreadSheet**
 VARIANT **ScreenContents**
 VARIANT **GridRedraw**
 ICwlGrid* **Grid**

BOOL **Active**

Methoden

long **GridLines** ()
 long **PushButton** (VARIANT PostIt)
 long **TreeExpand** (BOOL bAll)
 long **TreeCollapse** (BOOL bAll)
 long **TreeSelect** (BSTR strSearch, BOOL bSearchExact)
 long **ListboxSelect** (long nIndex)
 BOOL **SetCurrentGridCell** (long Row, long logColumn)
 BOOL **GetCurrentGridCell** (VARIANT * Row, VARIANT * logColumn)
 VARIANT **GetGridCellValue** (long Row, long logColumn)
 void **Validate** ()
 void **Refresh** ()
 BOOL **SetGridColReadOnly** (long logColumn, VARIANT bSet)
 BOOL **GetGridColReadOnly** (long logColumn)
 void **AddToSplitter** (long SplitterId, VARIANT bResize, VARIANT bTopLeft)

6.7.1. Eigenschaften

Id [short, read only]

ID des Elements, entspricht der ID im CTK.

Contents [VARIANT, read write]

Inhalt des Elements, abhängig von dessen Typ. Siehe dazu auch Konstanten - CWLControlTypes und die Eigenschaft **Type**.

Im Falle eines Grids ist dies der Wert der aktuellen Gridzelle.

Bei einem Button wird der Status ob der Button gedrückt ist zurückgegeben ("1"). Bei einem Button, der nicht gedrückt bleibt, ist das Ergebnis immer "0". Gesetzt wird der Status nicht mit der Zuweisung von "1" zum Button-Contents, sondern über die **PushButton** - Methode.

Beim Setzen dieses Parameters muß das Element das aktuelle Element sein. Das Setzen ruft implizit die Funktion **Validate** auf um der Applikation die Möglichkeit zu geben den gesetzten Wert zu verarbeiten, wodurch der Focus auf das nächste Element gesetzt wird.

Statische Felder müssen nicht das aktuelle Element im Fenster sein, weil dies ja gar nicht möglich ist, und bei diesen Feldern wird auch die Funktion **Validate** nicht aufgerufen.

Bei speziellen Datentypen (z.B. Datum) sollte der Wert der zugewiesen wird in den entsprechenden Typ konvertiert werden andernfalls wird nur eine interne Konversion ausgeführt, die nicht in allen Fällen zu korrekten Werten führt. Wird ein Datum als Text übergeben muß es auf jeden Fall im Format DD.MM.YYYY HH:MM:SS sein (die Zeit kann weggelassen werden).

Text [BSTR, read write]

Entspricht der Eigenschaft **Title** des Elements im CTK. Bei Comboboxen und PictureControls kann der Inhalt des Controls damit verändert werden.

Bei Comboboxen können die Einträge durch Strichpunkte getrennt übergeben werden. Die Eingekürzel sind vom Text mit : getrennt.

Bei Comboboxen ohne Eingabelänge (Eigenschaft **Letters** im CTK) muss trotzdem der : vor dem Eintrag stehen (z.B.: fg.Text = ":Erster;Zweiter").

Die Einträge werden in der Combobox-Liste immer am Ende angehängt. Um die Combobox neu zu füllen muss ein Leerstring übergeben werden. Im Anschluss können dann wieder Einträge hinzugefügt werden.

Beispiel:

```
fg.Text = "1:erster Eintrag;2:zweiter Eintrag"
```

View [long, read only]

Hinterlegte Programmvariable, View.

Var [long, read only]

Hinterlegte Programmvarianle, Var.

Line [long, read only]

Entspricht der Eigenschaft **Row** des Elements im CTK.

Column [long, read only]

Entspricht der Eigenschaft **Column** des Elements im CTK.

Type [CWLControlTypes, read only]

Typ des Elements, siehe auch Konstanten - CWLControlTypes.

Font [long, read only]

ID der verwendeten Schrift für dieses Element. Entspricht der Font ID im CTK.

Height [long, read only]

Entspricht der Eigenschaft **Height** des Elements im CTK.

Width [long, read only]

Entspricht der Eigenschaft **Width** des Elements im CTK.

Preview [ICwlPreview*, read only]

Pointer auf ein Objekt der Klasse **CWLPreview**, wenn es sich bei diesem Element um ein Preview Element handelt (Typ=cwlControlPreview) - siehe dazu auch Konstanten - CWLControlTypes und **CWLPreview**.

ScreenContents [VARIANT, read only]

Dies gibt den aktuellen Wert des Eingabefeldes zurück (ohne dass der Wert vom Programm validiert oder angepasst wird). Die Property **Contents** im Gegensatz dazu gibt den Inhalt der Variable zurück, mit der das Eingabefeld verknüpft ist (das bedeutet, dass während des OnCheck - Events nur **ScreenContents** den gerade aktuell eingegebenen Wert beinhaltet, die Variable wird erst nach dem Event vom Programm gefüllt und damit enthält **Contents** im OnCheck - Event noch den vorigen Wert.

Ausserhalb des OnCheck - Events sollten die beiden Eigenschaften den gleichen Wert beinhalten.

GridRedraw [VARIANT]

Bei größeren Änderungen in einer Bildschirmtable (Grid) kann sich Verarbeitung stark verlangsamen, wenn jede Veränderung am Bildschirm direkt angezeigt wird. Mit dieser Eigenschaft kann die Anzeige von Änderungen in der Bildschirmtable abgeschaltet werden, wird die Eigenschaft wieder auf True gesetzt, werden alle Änderungen auf einen Schlag angezeigt.

SpreadSheet [ICwlSpreadSheet*, read only]

Pointer auf ein Objekt der Klasse **CWLSpreadSheet**, wenn es sich bei diesem Element um ein SpreadSheet Element handelt (Typ=cwlControlSpreadsheet) - siehe dazu auch Konstanten - CWLControlTypes und **CWLSpreadSheet**.

Grid [ICwlGrid*, read only]

Pointer auf ein Objekt der Klasse **CWLGrid**, wenn es sich bei diesem Element um eine Bildschirmtablette handelt (Typ = cwlControlGrid) - siehe dazu auch Konstanten - CwlControlTypes und **CWLGrid**.

Active [BOOL, read write]

Gibt den aktuellen Status des Elements wieder (FALSE: ausgegraut, oder auf einem anderen Showlevel, TRUE: aktiv). Für selbst eingefügte Elemente kann der Status auch verändert werden.

6.7.2. Methoden**PushButton (VARIANT PostIt)**

Handelt es sich bei dem Element um ein PushButton (Typ= cwlControlButton), dann kann man mit dieser Methode den Push Event des Buttons feuern.

Das Element muß das aktuelle Element im Fenster sein, oder sich im Toolbar befinden, andernfalls erfolgt ein Runtime Error.

Der Parameter ist optional, und wird, wenn er nicht angegeben ist, mit FALSE übergeben. Damit wird die Funktion direkt ausgeführt. Wird TRUE übergeben, wird der Button erst gedrückt, wenn die aktuelle VB-Script Funktion beendet ist. Damit kann z.B. das aktuelle Fenster in einer VB-Script Funktion des Fensters geschlossen werden. Würde der Button direkt ausgeführt, würde dies zu einem Programmabsturz führen, weil die VB Scriptfunktion bei der Rückkehr von der Ausführung des Button-Clicks nichts mehr vorfinden würde - das Fenster wurde ja bereits geschlossen und damit die VB Script Anbindung beendet.

Parameter (optional)

FALSE	(default) Der Button-Click wird direkt ausgeführt
TRUE	Der Button-Click wird erst ausgeführt, wenn die aktuelle VB-Script Funktion beendet ist.

Rückgabewert (long)

Enthält den applikationsspezifischen Wert, der Wert ist standardmäßig 0, kann aber abhängig vom Fenster auch andere Werte annehmen.

TreeExpand (BOOL bAll)

Handelt es sich bei dem Element um einen Tree (Typ= cwlControlTree), dann kann man mit dieser Methode den aktuell selektierten Zweig eines Trees öffnen.

Abhängig vom Parameter **bAll** kann man auch den gesamten Tree expandieren.

Das Element muß das aktuelle Element im Fenster sein, andernfalls erfolgt ein Runtime Error.

Parameter

TRUE	Expandiert den gesamten Tree
FALSE	Expandiert nur den selektierten Ast um eine Stufe

Rückgabewert (long)

Enthält den applikationsspezifischen Wert, der Wert ist standardmäßig 0, kann aber abhängig vom Fenster auch andere Werte annehmen.

TreeCollapse(VARIANT BOOL bAll)

Handelt es sich bei dem Element um einen Tree (Typ= cwlControlTree), dann kann man mit dieser Methode den aktuell selektierten Zweig eines Trees schließen.

Abhängig vom Parameter **bAll** kann man auch den gesamten Tree schließen.
Das Element muß das aktuelle Element im Fenster sein, andernfalls erfolgt ein Runtime Error.

Parameter

TRUE	Schließt den gesamten Tree
FALSE	Schließt nur den selektierten Ast

Rückgabewert (long)

Enthält den applikationsspezifischen Wert, der Wert ist standardmäßig 0, kann aber abhängig vom Fenster auch andere Werte annehmen.

TreeSelect(BSTR strSearch, BOOL bSearchExact)

Handelt es sich bei dem Element um einen Tree (Typ= cwlControlTree), dann kann man mit dieser Methode ein Tree Element gesucht werden. Abhängig vom Parameter bSearchExact wird bei exakter Suche das Element nur dann gefunden, wenn der vollständige Text übergeben wurde, andernfalls genügt es wenn nur der Suchtext selbst übereinstimmt.

Das Element muß das aktuelle Element im Fenster sein, andernfalls erfolgt ein Runtime Error.

Parameter

strSearch	Der Text, der im Tree gesucht werden soll.
bSearchExact	Der gesuchte Text muß exakt übereinstimmen (TRUE) oder es muß der gefundene Tree Text mit strSearch beginnen (FALSE)

Rückgabewert (long)

Enthält den applikationsspezifischen Wert, der Wert ist standardmäßig 0, kann aber abhängig vom Fenster auch andere Werte annehmen.

GridLines

Handelt es sich bei dem Element um ein Grid (Typ=cwlControlGrid), dann liefert diese Methode die Anzahl der ansprechbaren Zeilen in diesem Grid. Andernfalls liefert die Methode 0.

Rückgabewert (long)

Anzahl der Grid Zeilen.

SetCurrentGridCell(long Row, long logColumn)

Setzt den Cursor in einem Grid auf die angegebene Zelle.

Das Element muß das aktuelle Element im Fenster sein, andernfalls erfolgt ein Runtime Error.

Parameter

Row	Zeile im Grid, beginnend mit "1"
logColumn	Logische Spaltennummer (unabhängig von der Benutzeranordnung), beginnen mit "1" Jede Spalte hat eine eindeutige logische Nummer, die sie unabhängig von der Position im Grid behält.

Rückgabewert (VARIANT_BOOL)

Wenn die Zelle gesetzt werden konnte ist das Ergebnis TRUE, andernfalls erfolgt ein Runtime Error.

GetCurrentGridCell(VARIANT Row, VARIANT logColumn)

Liefert in die Übergabeparameter die aktuelle logische Position des Cursors in einem Grid.

Parameter

Row	Aktuelle Zeile im Grid, beginnend mit "1"
logColumn	Aktuelle Logische Spaltennummer (unabhängig von der Benutzeranordnung), beginnen mit "1" Jede Spalte hat eine eindeutige logische Nummer, die sie unabhängig von der Position im Grid behält.

Rückgabewert (VARIANT_BOOL)

Wenn die aktuelle Zelle gefunden werden konnte ist das Ergebnis TRUE, andernfalls erfolgt ein Runtime Error.

GetGridCellValue(long Row, long logColumn)

Liefert den Wert der angegebenen Zelle im Grid. Bei Angabe einer ungültigen oder noch nicht befüllten Zelle wird ein Runtimeerror ausgelöst.

Der Wert der aktuellen Grid Zelle kann auch mit der Property **Contents** des FGControl Objekts ermittelt werden.

Parameter

Row	Zeile der Zelle im Grid, beginnend mit "1"
logColumn	Logische Spaltennummer der Zelle im Grid (unabhängig von der Benutzeranordnung), beginnen mit "1" Jede Spalte hat eine eindeutige logische Nummer, die sie unabhängig von der Position im Grid behält.

Rückgabewert (VARIANT)

Wert aus der Zelle.

Validate

Validiert das aktuelle Element (entsprechend wenn der Benutzer "Enter" drückt) und bewegt den Cursor zum nächsten Element. Innerhalb der Grid wird dies nur bei Gridzellen ausgeführt, die ein Editfeld oder eine Combobox enthalten (nur diese müssen validiert werden) und bewegt den Cursor zur nächsten Gridzelle.

Refresh

Refresh stellt das Element erneut am Bildschirm dar. Dies ist notwendig, wenn sich die Variable, die mit dem Element verknüpft ist, geändert hat.

SetGridColReadOnly(long logColumn, VARIANT bSet)

Diese Funktion setzt die angegebene Spalte read/only, oder hebt den read/only Status wieder auf.

Parameter

logColumn	Die Spalte, die verändert werden soll, die 'logische' Spalte beginnend mit "1"
bSet	Optionaler Parameter (per default TRUE), der angibt, ob die Spalte read/only werden soll oder nicht

Rückgabewert (VARIANT_BOOL)

Wenn der Status gesetzt werden kann, wird TRUE zurückgegeben, bei einem Fehler FALSE.

GetGridColReadOnly(long logColumn)

Diese Funktion prüft, ob die angegebene Spalte read/only ist.

Parameter

logColumn	Die Spalte, die verändert werden soll, die 'logische' Spalte beginnend mit "1"
-----------	--

Rückgabewert (VARIANT_BOOL)

Wenn der read/only - Status gesetzt ist, wird TRUE zurückgegeben, andernfalls FALSE.

AddToSplitter(long SplitterId, VARIANT bResize, VARIANT bTopLeft)

Diese Funktion fügt das Fg-Element einem Splitter hinzu, sodass das Element mit dem Splitter in der Größe verändert oder verschoben werden kann.

Parameter

SplitterId	Die Fg-Nummer des Splitters an den das Element gehängt werden soll
bResize	Optionaler Parameter (per default TRUE). Gibt an, ob das Element mit dem Splitter vergrößert/verkleinert (TRUE) oder mit dem Splitter verschoben werden soll (FALSE)
bTopLeft	Nur wenn der Parameter bResize auf TRUE gesetzt ist, kann damit definiert werden, ob das Element links bzw. oben fixiert ist und mit dem Splitter nach rechts bzw. unten vergrößert/verkleinert wird (TRUE) oder ob es rechts bzw. unten fixiert ist und mit dem Splitter nach oben bzw. links in der Größe verändert wird (FALSE)

6.8. CwlPreview

Ein Objekt dieser Klasse repräsentiert eine CWL Preview. Eine Preview ist eine spezielle Ausprägung eines Controls (CWLFGControl), auf die über dessen Property **Preview** zugegriffen wird.

Eine Preview besteht aus einer Collection von PreviewPages, welche wiederum eine Collection von PreviewPageItems enthält.

Die ID einer Preview, mit der Sie aus der **CWLFGCollection** referenziert werden kann, ist dynamisch und muß während der Laufzeit festgestellt werden.

Ein reines Previewfenster besteht aus einem Fenster mit genau einem PreviewControl.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLPreview

Eigenschaften

long **PageCount**

long **CurrentPageNr**

ICwlPreviewPage* **Page** (long nPageNr)

Print (BOOL bChoosePrinter)
Mail (BOOL bWithDialog)

6.8.1. Eigenschaften

PageCount [long, read only]

Anzahl der Seiten (**CWLPreviewPage** Objekte) in der Preview (**Page** Property).
Wenn die Preview gerade gefüllt wird, kann es passieren, daß dieser Wert noch nicht die endgültige Anzahl an Seiten enthält.

CurrentPageNr [long, read only]

Aktuelle Seite, die gerade von dieser Preview dargestellt wird.

6.8.2. Methoden

Page(long nPageNr)

Liefert ein Objekt der Klasse **CWLPreviewPage**, das die Seite mit der im Parameter angegebenen Nummer **nPageNr** repräsentiert.

Parameter

nPageNr	Nummer der Seite, beginnen mit "1"
---------	------------------------------------

Rückgabewert (ICWLPreviewPage*)

Pointer auf ein Objekt der Klasse CWLPreviewPage der entsprechenden Seite.

Print(BOOL bChoosePrinter)

Die Funktion druckt das aktuelle Dokument auf dem Drucker/Spooler aus. Wird für bChoosePrinter TRUE übergeben, dann wird der Druckauswahldialog angezeigt und es wird auf dem dort gewählten Drucker ausgedruckt. Wird bChoosePrinter mit FALSE übergeben, dann wird am Standarddrucker ausgedruckt und wenn der Ausdruck auf den Spooler umgelenkt ist, wird in den Spooler gedruckt.

Parameter

bChoosePrinter	Auswahldialog für Drucker anzeigen
----------------	------------------------------------

Rückgabewert (keiner)

Mail(BOOL bWithDialog)

Die Funktion versendet das aktuelle Dokument per Mail im aktuell eingestellten System-Mailformat. Wird für bWithDialog TRUE übergeben, dann wird der Maildialog zur Eingabe eines Empfängers und weiterer optionaler Angaben für den Versand angezeigt. Wird bWithDialog mit FALSE übergeben, dann muss in dem Dokument ein AUX:MAIL Befehl vorhanden sein, aus dem der Empfänger des Mails extrahiert werden kann. Ist dieser Eintrag nicht vorhanden, wird ein Fehler erzeugt.

Parameter

bWithDialog	Dialog des Mailproviders zur Eingabe eines Empfängers und weiterer optionalen Angaben für den Versand
-------------	---

Rückgabewert (keiner)

6.9. CwlPreviewPage

Ein Objekt dieser Klasse repräsentiert eine einzelne Seite in einer Preview. Sie besteht aus einer Reihe von PreviewPage Objekten der Klasse **CWLPreviewPageItem**.

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLPreviewPage

Eigenschaften

long **ItemCount**

ICwlPreviewPageItem* **Item** (long nIndex)

6.9.1. Eigenschaften

ItemCount [long, read only]

Anzahl der Elemente vom Typ **CWLPreviewPageItem** in dieser PreviewPage.

6.9.2. Methoden

Item(long nIndex)

Liefert ein einzelnes Element in dieser PreviewPage der Klasse **CWLPreviewPageItem** zurück.

Parameter

nIndex	Nummer des Elements in der PreviewPage (0 bis ItemCount-1)
--------	---

6.10. CwlPreviewPageItem

Repräsentiert ein einzelnes Element innerhalb einer PreviewPage (**CWLPreviewPage**).

Verfügbar in

- ☐ System Makros
- ☐ CTK Makros

CWLPreviewPageItem

Eigenschaften

BSTR **Text**

long **View**

long **Var**

long **Line**

long **Column**

CWLSpoolItemType **Type**

CWLAlignments **Alignment**

long **Font**
long **Height**
long **Width**
BSTR **HiddenText** (CWLSpoolPreviewItemFlag flag)

6.10.1. Eigenschaften

Text [BSTR, read only]

Inhalt des Elements.

View [long, read only]

Dem Element hinterlegte View (im PDF Editor festgelegt).

Var [long, read only]

Dem Element hinterlegte Var (im PDF Editor festgelegt).

Line [long, read only]

Zeile, in der das Element dargestellt wird.

Column [long, read only]

Spalte, in der das Element dargestellt wird.

Type [long, read only]

Typ des Preview Elements (siehe CWLSpoolItemType)

Alignment [long, read only]

Ausrichtung (siehe CWLAlignments).

Font [long, read only]

Schriftart.

Height [long, read only]

Höhe des Elements.

Width [long, read only]

Breite des Elements.

HiddenText (CWLSpoolPreviewItemFlag flag) [BSTR, read only]

Ergibt den dem flag entsprechenden Text (siehe CWLSpoolPreviewItemFlag). Im Normalfall enthalten nur DrillDown Elemente einen Text.

6.11. CWLSpreadSheet

CWLSpreadSheet

Eigenschaften

BSTR Contents
 long LineCount
 long ColumnCount
 BSTR Formula

Methoden

BOOL **SetCurrentCell** (long row, long col)
 BOOL **GetCurrentCell** (VARIANT *row, VARIANT *col)
Recalc
Redraw
 BOOL **ExportAsXLS** (BSTR NameAndPath)
 BOOL **Load** (BSTR NameAndPath)
 BOOL **Save** (BSTR NameAndPath)

6.11.1. Eigenschaften

Contents [BSTR]

Der Inhalt der aktuellen Zelle im Spreadsheet.

LineCount [long]

Die Anzahl der Zeilen im Spreadsheet.

ColumnCount [long]

Die Anzahl der Spalten im Spreadsheet.

Formula [BSTR]

Die Formel der aktuellen Zelle. Enthält die Zelle einen konstanten Wert, wird ein leerer Text zurückgegeben.

6.11.2. Methoden

SetCurrentCell (long row, long col)

Setzt den Focus auf die gewünschte Zelle.

Parameter

row	Die Zeile im Spreadsheet
col	Die Spalte im Spreadsheet

Rückgabewert (BOOL)

Gibt False zurück, falls die angegebene Zelle nicht existiert, andernfalls True.

GetCurrentCell (VARIANT *row, VARIANT *col)

Stellt die aktuelle Zelle im Spreadsheet fest und setzt die übergebenen Parameter row und col auf den entsprechenden Wert.

Parameter

row	Referenz auf eine Variable, die mit der Zeile im gefüllt wird
col	Referenz auf eine Variable, die mit der Spalte gefüllt wird

Rückgabewert (BOOL)

Gibt False zurück, falls das Spreadsheet noch nie angeklickt wurde und damit keine Zelle aktiv ist.

Recalc

Zwingt das Spreadsheet die Werte neu durchzurechnen.

Redraw

Erzwingt ein Neuzeichnen des Spreadsheets.

SaveAsXLS (BSTR NameAndPath)

Exportiert das Spreadsheet im XLS Format. Dabei gehen alle Formeln und Einstellungen verloren, die nicht kompatibel zu Excel sind.

Parameter

NameAndPath	Der Name der Zielfeile mit Pfad.
-------------	----------------------------------

Rückgabewert (BOOL)

Gibt False zurück, falls das Spreadsheet nicht exportiert werden konnte.

Load (BSTR NameAndPath)

Lädt ein vorher im eigenen Format gespeichertes Spreadsheet (Save). Der Zustand des Spreadsheets der beim Speichern aktiv war, wird exakt wieder hergestellt.

Parameter

NameAndPath	Der Name der Zielfeile mit Pfad.
-------------	----------------------------------

Rückgabewert (BOOL)

Gibt False zurück, falls das Spreadsheet nicht geladen werden konnte.

Save (BSTR NameAndPath)

Exportiert das Spreadsheet im eigenen internen Format. Dieses Format kann später auch wieder eingelesen werden und erzeugt ein exaktes Abbild des Spreadsheets zum Zeitpunkt des Speicherns.

Parameter

NameAndPath	Der Name der Zielfeile mit Pfad.
-------------	----------------------------------

Rückgabewert (BOOL)

Gibt False zurück, falls das Spreadsheet nicht gespeichert werden konnte.

6.12. CWLGrid

CWLGrid
Eigenschaften
VARIANT Contents
long LineCount
long ColumnCount
BOOL IsRedraw
VARIANT SelectedLines
Methoden
BOOL SetCurrentCell (long row, long col)
BOOL GetCurrentCell (VARIANT *row, VARIANT *col)
BOOL ExportAsXLS (BSTR NameAndPath)
BOOL Load (BSTR Settings)
BOOL Save (BSTR Settings)
VARIANT GetCellValue (long row, long col)
BOOL GetColumnReadOnly (long col)
SetColumnReadOnly (long col, VARIANT bSet, VARIANT bRedraw)
long AddColumn (BSTR ColumnTitle, BSTR ColumnControl, BSTR align, BSTR Type, int Font, int View, int Var, int ColWidth, VARIANT AddFlags, VARIANT ColumnColor, VARIANT bRedraw)
BOOL RemoveColumn (long col, VARIANT bRedraw)
SetFooterColumn (int ColumnNr, BSTR ColumnControl, BSTR Align, BSTR Type, int Font, int View, int Var, VARIANT bRedraw)
SetColumnColor (long col, RGB color)
RGB GetColumnColor (long col)
SetLineColor (long line, RGB color)
RGB GetLineColor (long line)
BOOL MoveColumn (long col, long Position)
BOOL SetColumnWidth (long col, long Width)
long GetColumnWidth (long col)
long GetLogColumn (long ColumnOnScreen)
long GetPhysColumn (long col)
SetComboStrings (long col, BSTR theStrings)
Validate
Refresh
BOOL IsUserColumn (long col)
BOOL Header
BOOL Footer
BOOL AddLine
BOOL RemoveLine (long line)
BOOL InsertLine (long line)
BOOL ReplaceLine (long line)
GetLineValues (long line)
BOOL InitUserGrid
BOOL SetColumnTitle (long line, long col, BSTR Text)
SetDecimalPlaces (long line, long col, short places)
SetCellValue (long line, long col, VARIANT Value)
SetDrillDown (long line, long col, VARIANT DrillDown)
VARIANT GetDrillDown (long line, long col)
Clear (VARIANT where)
UpdateVars (long line, long logColumn)

6.12.1. Eigenschaften

Contents [VARIANT]

Der Inhalt der aktuellen Zelle.

LineCount [long]

Die Anzahl der Zeilen.

ColumnCount [long]

Die Anzahl der Spalten.

IsRedraw [BOOL]

Status ob Änderungen in der Bildschirmtabelle auch angezeigt werden (wird gesetzt um mehrere Änderungen durchzuführen und erst am Ende die Bildschirmausgabe durchzuführen).

SelectedLines [VARIANT]

Ist ein Array mit den ausgewählten, farblich markierten Zeilen. Bei Bildschirmtabellen, die keine Mehrfachselektion zulassen, kann immer nur eine Zeile ausgewählt (markiert) sein.

Wenn man der Eigenschaft Werte zuweist, kann das entweder ein Array mit den gewünschten Zeilennummern sein, oder eine einzelne Zeilennummer.

Beim Auslesen der Eigenschaft wird immer ein Array zurückgegeben.

Bevor man die Eigenschaft auswertet, sollte man prüfen ob tatsächlich Zeilen selektiert sind, indem man die Eigenschaft mit IsArray() testet.

Beispiel:

```
Set grid = CWLCurrentWindow.ActiveWindow.Controls.Item(100).Grid
theArray = grid.SelectedLines
If isarray (theArray) Then
    members = Ubound(theArray) + 1 & " Zeilen sind selektiert: "
    For i = 0 To Ubound(theArray)
        If i <> 0 Then members = members & ","
        members = members & theArray(i)
    Next
    msgbox members
Else
    msgbox "Keine Zeilen ausgewählt!"
End If

Dim fixArray
fixArray = Array (1,5,6,1000)
grid.SelectedLines = fixArray
```

6.12.2. Methoden

BOOL SetCurrentCell (long row, long col)

Setzt den Focus auf die gewünschte Zelle.

Parameter

row	Die Zeile in der Bildschirmtabelle
col	Die logische Spaltennummer

Rückgabewert (BOOL)

Gibt False zurück, falls die angegebene Zelle nicht existiert, andernfalls True.

BOOL GetCurrentCell (VARIANT *row, VARIANT *col)

Stellt die aktuelle Zelle in der Bildschirmtabelle fest und setzt die übergebenen Parameter row und col auf den entsprechenden Wert.

Parameter

row	Referenz auf eine Variable, die mit der Zeile im gefüllt wird
col	Referenz auf eine Variable, die mit der Spalte gefüllt wird

Rückgabewert (BOOL)

Gibt False zurück, falls die aktuelle Zelle durch einen internen Fehler nicht festgestellt werden konnte.

BOOL ExportAsXLS (BSTR NameAndPath)

Exportiert die Bildschirmtabelle im XLS Format.

Parameter

NameAndPath	Der Name der Zielfeile mit Pfad.
-------------	----------------------------------

Rückgabewert (BOOL)

Gibt False zurück, falls die Bildschirmtabelle nicht exportiert werden konnte.

BOOL Load (BSTR Settings)

Lädt die Gesamteinstellungen der Bildschirmtabelle, die zuvor unter dem Namen **Settings** gespeichert wurden.

Parameter

Settings	Der Name der gespeicherten Einstellungen
----------	--

Rückgabewert (BOOL)

Gibt False zurück, falls die Einstellungen nicht geladen werden konnten.

BOOL Save (BSTR Settings)

Speichert die Gesamteinstellungen der Bildschirmtabelle unter der Bezeichnung, die in **Settings** angegeben wird. Falls Einstellungen mit diesem Namen bereits existieren, werden sie überschrieben.

Parameter

Settings	Der Name der zu speichernden
----------	------------------------------

	Einstellungen
--	---------------

Rückgabewert (BOOL)

Gibt False zurück, falls die Einstellungen nicht gespeichert werden konnten.

VARIANT GetCellValue (long row, long col)

Es wird der Wert an der Stelle **row** (Zeile) und **col** (Spalte) zurückgegeben.

Parameter

row	Die Zeile in der Bildschirmtabelle
col	Die logische Spaltennummer

Rückgabewert (VARIANT)

Der Wert, der an der angegebenen Position in der Bildschirmtabelle gespeichert ist.

BOOL GetColumnReadOnly (long col)

Gibt den read-only Status der angegebenen Spalte (**col**) zurück.

Parameter

col	Die logische Spaltennummer
-----	----------------------------

Rückgabewert (BOOL)

Gibt False oder True zurück.

SetColumnReadOnly (long col, VARIANT bSet, VARIANT bRedraw)

Die Spalte col wird auf read-only gesetzt. Spalten, die diesen Typ haben, werden in einer eigenen Farbe angezeigt und können nicht mehr ausgewählt werden.

Parameter

col	Die logische Spaltennummer
bSet	read-only setzen oder nicht (optional, der Wert ist true, wenn er nicht angegeben wird)
bRedraw	die Änderung sofort am Bildschirm anzeigen (optional, der Wert ist true, wenn er nicht angegeben wird)

long AddColumn (BSTR ColumnTitle, BSTR ColumnControl, BSTR align, BSTR Type, int Font, int View, int Var, int ColWidth, VARIANT AddFlags, VARIANT ColumnColor, VARIANT bRedraw)

Fügt am Ende der Bildschirmtabelle eine neue Spalte hinzu. Eine Bildschirmtabelle kann maximal 199 Spalten haben, ist die Grenze erreicht, können keine neuen Spalten hinzugefügt werden.

Parameter

ColumnTitle	Der Titel der Spalte
ColumnControl	Text, der das Control beschreibt, das in der Zelle angezeigt werden soll (siehe Tabelle der möglichen Controls weiter unten).
align	Ausrichtung der Spalte: l... linksbündig r... rechtsbündig z... zentriert
Type	Der Typ der Zelle:

	T... reiner Anzeigetext V... Variable, die in der Zelle angezeigt wird G... eine Grafik)
Font	Die Nummer der Fontkombination (mögliche Werte sind von 0 bis 9). Die Fonts können im CWLCTK im Menüpunkt 'Edit Mesonic Fonts' im Reiter 'Other Fonts' verändert werden.
View	Die Tabelle (oder 0) aus der die angezeigte Variable kommt
Var	Die Nummer der Variable innerhalb der View .
ColWidth	Die Breite der Spalte in Bildeinheiten.
AddFlags	Eine Kombination von Werten, die das Verhalten der Spalte steuert: SORTFLAG = 1 (ist die Spalte sortierbar) HIDEFLAG = 4 (kann die Spalte versteckt werden) READONLYFLAG = 8 (ist die Spalte read-only) MOVEFLAG = 16 (kann die Spalte verschoben werden) SIZEFLAG = 32 (kann die Spalte in der Größe verändert werden) INVISIBLEFLAG = 64 (Spalte kann nicht angezeigt werden) COMPANYYEARFLAG = 256 (Spalte enthält das Wirtschaftsjahr, und wird bei Verwendung eines anderen Kalenders automatisch in diesen umgerechnet) Der Wert ist optional und 0 wenn er nicht angegeben wird.
ColumnColor	Optional eine eigene Spaltenfarbe. Wenn nicht angegeben, erhält die Spalte keine eigene Farbe (ausgenommen sie ist read-only).
Redraw	Optionale Angabe, ob die Änderung sofort am Bildschirm angezeigt werden soll. Wenn nicht angegeben, ist der Wert true. Wird false übergeben, muss zu einem späteren Zeitpunkt die Methode Refresh aufgerufen werden.

Mögliche Controls für eine Zelle in der Bildschirmtabelle (Voraussetzung: Spaltentyp "V"):

Typ	Control	Variablentyp	Mögliche Parameter	Beispiel
T1	Eingabefeld	Text	Zx (maximale Zeichenzahl) L1 (Matchcode Lupe) Ox (Objektyp)	"T1,Z10,L1,Mein Eingabefeld"
T2	Eingabefeld	Integer	Zx (maximale Zeichenzahl) Ox (Objektyp)	"T2,Z5,Mein Eingabefeld"

T3	Eingabefeld	Double	Zx (maximale Zeichenanzahl) Ix (Anzahl Nachkommastellen) L1 (Matchcode Lupe) Ox (Objektyp)	"T3,Z15,I2,L1,Mein Eingabefeld"
T5	Eingabefeld	Großbuchstaben	Zx (maximale Zeichenanzahl) L1 (Matchcode Lupe) Ox (Objektyp)	"T5,Z10,L1,Mein Eingabefeld"
T6	Eingabefeld	Datum	Zx (maximale Zeichenanzahl) L1 (Matchcode Lupe) Ox (Objektyp)	"T6,Z10,L1,Mein Eingabefeld"
T12	Checkbox	Text	Z1 Ox (Objektyp)	"T12,Z1,Meine Checkbox"
T17	read-only Checkbox	Text	Ox (Objektyp)	"T12,Z1,Meine Checkbox"
T21	Statisch	Text	Ox (Objektyp)	"T21,Mein Wert"*
T22	Statisch	Integer	Ox (Objektyp)	"T22,Mein Wert"*
T23	Statisch	Double	Ox (Objektyp) Ix (Anzahl Nachkommastellen)	"T23,Mein Wert"*
T25	Statisch	Großbuchstaben	Ox (Objektyp)	"T25,Mein Wert"*
T26	Statisch	Datum	Ox (Objektyp)	"T26,Mein Wert"*
T31	Combobox	Text	Zx (Eingabelänge) Lx (Breite der Listbox, bei 0 wird die Spaltenbreite verwendet) Hx (Höhe der Listbox)	"T31,Z1,L0,H5,Meine Combo"
T31	Combobox mit Mehrfachauswahl	Text	Z0 (Eingabelänge muss 0 sein) Lx (Breite der Listbox, bei 0 wird die Spaltenbreite verwendet) I1 (Aktivierung Mehrfachauswahl) Hx (Höhe der Listbox)	"T31,Z0,L0,I1,H5,Meine Combo"
T32	read-only Combobox	Text	Zx (Eingabelänge)	"T32,Z1,Meine read-only Combo"

* Für die Darstellung von statischen Werten ist es oft günstiger nicht den Typ anzugeben (z.B. T21), sondern direkt das Format für die Ausgabe, wie es auch im PDF-Editor verwendet wird (z.B. "{DATETIME}" bei einem Datum).

Rückgabewert (long)

Die logische Nummer der angefügten Spalte, oder 0 falls das Anfügen scheiterte.

BOOL RemoveColumn (long col, VARIANT bRedraw)

Entfernt eine zuvor eingefügte Spalte. Spalten, die nicht selbst eingefügt wurden, können nicht entfernt werden.

Parameter

col	Die logische Spaltennummer
Redraw	Optionale Angabe, ob die Änderung sofort am Bildschirm angezeigt werden

	soll. Wenn nicht angegeben, ist der Wert true. Wird false übergeben, muss zu einem späteren Zeitpunkt die Methode Refresh aufgerufen werden.
--	--

Rückgabewert (BOOL)

Gibt False oder True zurück, abhängig davon ob die Spalte entfernt wurde.

SetFooterColumn (int ColumnNumber, BSTR ColumnControl, BSTR align, BSTR Type, int Font, int View, int Var, VARIANT bRedraw)

Setzt für eine Spalte im Fuß die Beschreibung, die beim Ausgeben des Fußes für die Anzeige verwendet wird.

Wie in der **AddColumn** - Funktion können auch hier die einzelnen Parameter für eine Zelle angegeben werden.

Parameter

ColumnNumber	Die logische Spaltennummer
ColumnControl	Text, der das Control beschreibt, das in der Zelle angezeigt werden soll (siehe Tabelle der möglichen Controls bei der AddColumn - Funktion).
align	Ausrichtung der Spalte: l... linksbündig r... rechtsbündig z... zentriert
Type	Der Typ der Zelle: T... reiner Anzeigetext V... Variable, die in der Zelle angezeigt wird G... eine Grafik)
Font	Nummer des Fonts aus der mesocol.ini
View	Die Tabelle (oder 0) aus der die angezeigte Variable kommt
Var	Die Nummer der Variable innerhalb der View .
Redraw	Optionale Angabe, ob die Änderung sofort am Bildschirm angezeigt werden soll. Wenn nicht angegeben, ist der Wert true. Wird false übergeben, muss zu einem späteren Zeitpunkt die Methode Refresh aufgerufen werden.

SetColumnColor (long col, RGB color)

Die Spalte **col** erhält die Farbe **color**. Die Farbe wird im RGB-Format übergeben.

Parameter

col	Die logische Spaltennummer
color	Die gewünschte Farbe im RGB-Format (in VBScript kann dafür die Funktion RGB verwendet werden: RGB(Rotwert,Grünwert,Blauwert))

Es wird die Farbe der Spalte **col** im RGB-Format zurückgegeben. Ist keine eigene Farbe gesetzt, wird -1 zurückgegeben.

Parameter

col	Die logische Spaltennummer
-----	----------------------------

Rückgabewert (RGB)

Die Farbe der Spalte. Die rot/blau/grün - Komponenten des Farbwerts kann mit folgender Funktion bestimmt werden:

```
If (color >= 0) Then
    blue = color\65536
    green = (color-(blue*65536))\256
    red = color-(blue*65536) - (green*256)
End If
```

SetLineColor (long line, RGB color)

Die Zeile **line** erhält die Farbe **color**. Die Farbe wird im RGB-Format übergeben.

Parameter

line	Die logische Spaltennummer
color	Die gewünschte Farbe im RGB-Format (in VBScript kann dafür die Funktion RGB verwendet werden: RGB(Rotwert,Grünwert,Blauwert))

RGB GetLineColor (long col)

Es wird die Farbe der Zeile **line** im RGB-Format zurückgegeben. Ist keine eigene Farbe gesetzt, wird -1 zurückgegeben.

Parameter

col	Die logische Spaltennummer
-----	----------------------------

Rückgabewert (RGB)

Die Farbe der Spalte. Die rot/blau/grün - Komponenten des Farbwerts kann mit folgender Funktion bestimmt werden:

```
If (color >= 0) Then
    blue = color\65536
    green = (color-(blue*65536))\256
    red = color-(blue*65536) - (green*256)
End If
```

BOOL MoveColumn (long col, long Position)

Verschiebt die angegebene Spalte (**col**) auf die Position **Position**.

Parameter

col	Die logische Spaltennummer
Position	Die Zielposition (1 bis Anzahl der Spalten) an der die Spalte eingefügt werden soll.

Rückgabewert (BOOL)

Gibt False oder True zurück.

BOOL SetColumnWidth (long col, long Width)

Verändert die Breite der Spalte **col** auf die Breite **Width** (in Bildschirmeinheiten).

Parameter

col	Die logische Spaltennummer
Width	Die Spaltenbreite in Bildschirmeinheiten. Wird 0 übergeben, wird die Spalte versteckt.

Rückgabewert (BOOL)

Gibt False oder True zurück.

long GetColumnWidth (long col)

Die Funktion liefert die Breite der Spalte **col** in Bildschirmeinheiten.

Parameter

col	Die logische Spaltennummer
-----	----------------------------

Rückgabewert (long)

Die Breite der Spalte in Bildschirmeinheiten.

long GetLogColumn (long ColumnOnScreen)

Die Funktion liefert die logische Spaltennummer der **ColumnOnScreen** - ten Bildschirmspalte.

Parameter

ColumnOnScreen	Die Spaltenposition am Bildschirm
----------------	-----------------------------------

Rückgabewert (long)

Die logische Spaltennummer.

long GetPhysColumn (long col)

Die Funktion liefert die Position der Spalte **col** am Bildschirm.

Parameter

col	Die logische Spaltennummer
-----	----------------------------

Rückgabewert (long)

Die Position der Spalte am Bildschirm.

SetComboStrings (long col, BSTR theStrings)

Die Funktion setzt für eine Spalte mit Comboboxen, die möglichen Einträge der Combobox. Die Einträge werden in einem einzigen String übergeben, wo die Einträge durch Zeilenumbrüche von einander getrennt sind.

Ist die Combobox ohne Eingabelänge definiert, müssen nur die Texte mit CR/LF getrennt eingefügt werden (die Eingabelänge beschreibt die Länge des Wertes der Combobox. Dieser Wert wird in der Variable gespeichert, die die Combobox anzeigt. Der Anzeigetext ist ein Beschreibungstext für den Wert).

Parameter

col	Die logische Spaltennummer
-----	----------------------------

theStrings	String mit den Einträgen der Combobox. Der String hat folgendes Format: Wert<Tab>Anzeigetext<CR><LF> Wert<Tab>Anzeigetext<CR><LF> ...
------------	---

Beispiel:

```

combostring = "0"&chr(9)&"Option 0"&chr(13)&chr(10)
combostring = combostring & "1"&chr(9)&"Option 1"&chr(13)&chr(10)
combostring = combostring & "2"&chr(9)&"Option 2"&chr(13)&chr(10)
myGrid.SetComboStrings 14, combostring

```

Validate

Wir der Text einer Zelle (z.b. mit einem Makrobefehl geändert), wird mit diesem Befehl die Prüfung des Eingabewerts angestoßen, die auch das Event **OnGridCheckUserColumn** auslöst.

Refresh

Diese Funktion erzwingt ein Neuzeichnen der Bildschirmtabelle.

BOOL IsUserColumn (long col)

Die Funktion stellt fest, ob die Spalte **col** eine benutzerdefinierte Spalte ist, die nachträglich dem Grid hinzugefügt wurde.

Parameter

col	Die logische Spaltennummer
-----	----------------------------

Rückgabewert (BOOL)

Ob die Spalte eine benutzerefinierte Spalte ist.

BOOL Header

Die Funktion gibt den Kopf der Grid aus.

Die Funktion kann nur in selbst definierten Grids verwendet werden.

BOOL Footer

Die Funktion gibt den Fuß der Grid aus.

Die Funktion kann nur in selbst definierten Grids verwendet werden.

BOOL AddLine

Diese Funktion fügt eine neue Zeile am Ende der Grid hinzu. Die Spalten erhalten die Werte, die in den damit verbundenen Variablen gespeichert sind.

Welche Variablen in der Grid angezeigt werden, wird bei der Definition einer Spalte bestimmt.

Die Funktion kann nur in selbst definierten Grids verwendet werden.

Rückgabewert (BOOL)

Bei einem Fehler wird FALSE zurückgegeben, sonst TRUE.

BOOL RemoveLine (long line)

Diese Funktion entfernt die Zeile mit der Nummer **line**.

Die Funktion kann nur in selbst definierten Grids verwendet werden.

Parameter

line	Die Zeilennummer
------	------------------

Rückgabewert (BOOL)

Bei einem Fehler wird FALSE zurückgegeben, sonst TRUE.

BOOL InsertLine (long line)

Diese Funktion fügt eine neue Zeile vor der Zeile mit der Nummer **line** ein. Die Werte für die Spalten werden den mit den Spalten verbundenen Variablen entnommen.

Die Funktion kann nur in selbst definierten Grids verwendet werden.

Parameter

line	Die Zeilennummer
------	------------------

Rückgabewert (BOOL)

Bei einem Fehler wird FALSE zurückgegeben, sonst TRUE.

BOOL ReplaceLine (long line)

Diese Funktion ersetzt die Zeile mit der Nummer **line** durch eine neue Zeile. Die Werte für die Spalten werden den mit den Spalten verbundenen Variablen entnommen.

Die Funktion kann nur in selbst definierten Grids verwendet werden.

Parameter

line	Die Zeilennummer
------	------------------

Rückgabewert (BOOL)

Bei einem Fehler wird FALSE zurückgegeben, sonst TRUE.

GetLineValues (long line)

Diese Funktion kopiert die Spaltenwerte der Zeile mit der Nummer **line** in die mit den Spalten verbundenen Variablen.

Die Funktion kann nur in selbst definierten Grids verwendet werden.

Parameter

line	Die Zeilennummer
------	------------------

BOOL InitUserGrid

Diese Funktion initialisiert das Gridobjekt und verbindet die Variablen des Fensters in der die Grid definiert ist, mit der Grid.

Erst wenn diese Funktion erfolgreich aufgerufen wurde, kann das Gridobjekt verwendet werden.

Die Funktion kann nur in selbst definierten Grids verwendet werden.

Rückgabewert (BOOL)

Bei einem Fehler wird FALSE zurückgegeben, sonst TRUE.

BOOL SetColumnTitle(long line, long col, BSTR Text)

Mit dieser Funktion kann der Text einer Spalte im Kopf geändert werden.

Ist das Element mit einer Variable verknüpft, darf der Text maximal die Länge dieser Variable haben, alles was länger ist wird abgeschnitten. Der neue Text wird dabei in die Variable kopiert.

Parameter

line	Die Zeilennummer, im Kopf normalerweise 1
col	Die logische Spalte für die der Kopftext verändert wird
Text	Der Text, der in der Spalte im Kopf dargestellt werden soll

Rückgabewert (BOOL)

Bei einem Fehler wird FALSE zurückgegeben, sonst TRUE.

SetDecimalPlaces(long line, long col, short places)

Mit dieser Funktion kann die Anzahl der Nachkommastellen für ein Zahleneingabefeld geändert werden.

Parameter

line	Die Zeilennummer
col	Die logische Spalte
places	Die Anzahl der Nachkommastellen

SetCellValue(long line, long col, VARIANT Value)

Mit dieser Funktion kann ein beliebiges Feld in der Grid mit einem neuen Wert gesetzt werden. Dies kann nur für vom Benutzer definierte Spalten ausgeführt werden, da das Programm von der Veränderung nichts erfährt.

Parameter

line	Die Zeilennummer
col	Die logische Spalte
Value	Der Wert, der in der Zelle gesetzt werden soll (der Typ muss mit dem Zellentyp kompatibel sein)

SetDrillDown(long line, long col, VARIANT DrillDown)

Mit dieser Funktion können benutzerdefinierte Spalten als Drilldownspalten gesetzt werden, wobei entweder ein eigener Text oder ein Objekttyp übergeben werden kann. Wird ein eigener Text vergeben, dann kann dies nur im neuen OnGridDrillDown - Event behandelt werden. Nur statische Felder können als Drilldownfelder markiert werden und reagieren dann auf einen Mausklick.

Wird ein Objekttyp hinterlegt (oder der Defaultobjekttyp, der auf den Objekttyp des Wertes in der Grid reagiert), dann stehen die Standard-Drilldown Operationen zur Verfügung.

Es kann die gesamte Spalte mit einem Drilldown versehen werden, wobei alle Zellen den gleichen Drilldownwert hinterlegt haben. Dies wird erreicht, indem die Zeilennummer mit 0 gesetzt wird. Diese Aktion hat im Normalfall nur dann einen Sinn, wenn die Zellen mit einem Objekttyp hinterlegt wurden, und das Drilldown den Wert „@USEDEFAULTOBJECT“ erhält. Damit reagiert dann jede Zelle abhängig vom darin enthaltenen Objekt (z.B. ein Artikel) mit der zu dem Objekt gehörenden Aktion.

Der übergebene Wert (Value) kann verschiedene Bedeutungen haben:

- nicht vorhanden oder true: das oben erwähnte @USEDEFAULTOBJECT wird hinterlegt.
- false, oder 0 oder ein leerer Text: das Drilldown wird wieder weggenommen.
- > 1 (numerisch): der Wert wird als Objekttyp interpretiert und es wird @MESOOBJECT[wert] hinterlegt
- Text: der Text wird als Drilldowntext hinterlegt

Wird ein beliebiger Text als Drilldown hinterlegt, dann muss auf den Klick mit dem OnGridDrillDown - Event reagiert werden. Es gibt dann kein Defaultverhalten.

Parameter

line	Die Zeilennummer
col	Die logische Spalte
Value	Der Wert, der in der Zelle als Drilldownwert hinterlegt wird

Hinweis:

Die folgenden Werten stehen als "Value" (=Objektyp) zur Verfügung:

- ☐ 5: Sachkonten
- ☐ 50: Personenkonten
- ☐ 21: Artikel
- ☐ 324: Produktionsaufträge
- ☐ 60: Projekte
- ☐ 400: Arbeitnehmer (A)
- ☐ 45: Kontakte und Ansprechpartner
- ☐ 51: Interessenten
- ☐ 170: CRM

VARIANT GetDrillDown (long line, long col)

Mit dieser Funktion kann der Drilldownwert, der in einer Zelle (oder einer ganzen Spalte) hinterlegt ist, abgefragt werden.

Wird als Zeile (line) 0 übergeben, wird ein eventuell für die gesamte Spalte vergebener Drilldowntext zurückgegeben.

Ist kein Drilldownwert hinterlegt wird ein leerer Text zurückgegeben.

Parameter

line	Die Zeilennummer
col	Die logische Spalte

Rückgabewert (VARIANT)

Der Drilldowntext.

Clear (VARIANT Where)

Es wird der angegebene Bereich der Grid gelöscht. Wird **Where** nicht angegeben wird das gesamte Grid gelöscht (Kopf, Mittelteil und Fuß).

Parameter

Where	(optional) Der Bereich, der gelöscht werden soll: 1001... Kopf 1002... Mittelteil 1003... Fuß 1004... Alles
-------	--

Beispiel:

Das Beispiel geht von einem angepassten Fenster aus, in dem eine Grid mit der ID 100 enthalten ist.

Mit verschiedenen Buttons werden in der Grid:

- eine Spalte hinzugefügt, die Spalte eingefärbt und an die dritte Bildschirmposition verschoben
- die Spalte wieder gelöscht
- die Grideinstellungen gespeichert und geladen
- der Gridinhalt in einer Exceltabelle gespeichert
- grundsätzliche Grideigenschaften ausgelesen

UpdateVars (long line, long logColumn)

Mit dieser Funktion wird der Wert in der Spalte (line/logColumn) in die zugehörige Programmvariable kopiert, die beim Einfügen der Spalte angegeben worden ist.

Die Funktion kann nur auf Spalten angewendet werden, die selbst hinzugefügt wurden, nicht für Spalten, die bereits im Programm vorhanden sind.

Wird als Zeile (line) oder Spalte (logColumn) 0 übergeben, wird der Wert durch die Zeile/Spalte der aktuellen Position in der Tabelle ersetzt.

Wird die Spalte (logColumn) mit -1 übergeben, werden alle benutzerdefinierten Spalten der Zeile behandelt.

Parameter

Line	Die Zeilennummer (0 bedeutet aktuelle Zeile)
logColumn	Die logische Spalte (0 bedeutet aktuelle Spalte, -1 bedeutet alle Benutzerspalten)

Rückgabewert: keiner

Beispiel:

```
' Event, das beim Verlassen einer (User-)Gridspalte gefeuert wird
Sub CWLCurrentWindow_OnGridCheckUserColumn(nFgId, nRow, nColumn, bResult)
    If nFgId = 100 Then
        Set myGrid = CWLCurrentWindow.ActiveWindow.Controls.Item(100).Grid

        ' In Zeilen <= 5 darf der Wert "2" nicht verwendet werden
        If myGrid.Contents = "2" And nRow <= 5 Then
            General.MsgBox "Nur die Werte 0 und 1 sind In den Zeilen 1 bis
5 erlaubt!"
            bResult.value = False
        End If
    End If
End Sub

' Pushbutton Events
Sub CWLCurrentWindow_OnPushButton(nFgId, bResult)

    Dim row, column
    Set myGrid = CWLCurrentWindow.ActiveWindow.Controls.Item(100).Grid

    If nFgId = 800 Then ' Pushbutton 'Spalte hinzufügen'
        If myGrid.ColumnCount = 13 Then ' Spalte ist noch nicht dazugefügt
worden?
            myGrid.isRedraw = 0 ' Änderungen nicht gleich anzeigen

            ' die neue Spalte ist eine Combobox mit Eingabelänge 1
            ' Liste der Combоеinträge erzeugen
            combostring = "0"&chr(9)&"Option 0"&chr(13)&chr(10)
            combostring = combostring & "1"&chr(9)&"Option
1"&chr(13)&chr(10)
            combostring = combostring & "2"&chr(9)&"Option
2"&chr(13)&chr(10)
            myGrid.SetComboStrings 14, combostring
```

```

' Neue Variable für die Gridspalte in den benutzerdefinierten
Variablen erzeugen
CWLCurrentWindow.ActiveWindow.Vars.CreateVar 495, 0, "1", 1,
"1"

' Spalte hinzufügen
myColumnNumber = myGrid.AddColumn ("Meine Spalte",
"T31,Z1,L30,H3,mycombo","l", "V", 0, 495, 0, 20)

' Spalte an die dritte Position setzen
myGrid.MoveColumn myColumnNumber,3

' Spaltenfarbe der neuen Spalte verändern
myGrid.SetColumnColor myColumnNumber, RGB(177, 200, 233)
' Zeilenfarbe der Zeilen 5,7,9 und 11 verändern
myGrid.SetLineColor 5, RGB(222, 232, 245)
myGrid.SetLineColor 7, RGB(222, 232, 245)
myGrid.SetLineColor 9, RGB(222, 232, 245)
myGrid.SetLineColor 11, RGB(222, 232, 245)

myGrid.isRedraw = 1 ' veranlasst ein Neuzeichnen

' !! Darf erst nach dem isRedraw = 1 gemacht werden, da
' während des unterbundenen Redraws keine Controls erzeugt
' werden und damit die eingefügte Combo nicht vorhanden wäre,
' was das SetContents scheitern ließe
' Focus auf die 3. Zeile in die neue Spalte
myGrid.SetCurrentCell 3, myColumnNumber

' Zellenwert auf "2" setzen ==> provoziert einen Fehler in
OnGridCheckUserColumn
myGrid.Contents = "2"

End If
End If

If nFgId = 799 Then ' Button 'Spalte entfernen'
    If myGrid.ColumnCount = 14 Then
        myGrid.RemoveColumn 14
    End If
End If

If nFgId = 798 Then ' Button 'Einstellungen laden'
    myGrid.load "MDP Settings"
End If

If nFgId = 797 Then ' Button 'Einstellungen speichern'
    myGrid.save "MDP Settings"
End If

If nFgId = 796 Then ' Button 'nach Excel exportieren'
    myGrid.ExportAsXLS "c:\mdp script grid.xls"
End If

```



```

If nFgId = 795 Then      ' Button 'Grid Infos auslesen'
    msg = "Grid Informationen:" & chr(13) & chr(10)
    msg = msg & "Zeilen: " & myGrid.LineCount & chr(13) & chr(10)
    msg = msg & "Spalten: " & myGrid.ColumnCount & chr(13) & chr(10)
    myGrid.GetCurrentCell row, column
    msg = msg & "aktuelle Zelle: " & row & "/" & column & chr(13) &
chr(10)
    msg = msg & "Zelleninhalt: " & myGrid.Contents & chr(13) & chr(10)
    msg = msg & "logische Spaltennummer: " & column & chr(13) & chr(10)
    msg = msg & "sichtbare Spaltennummer: " & myGrid.GetPhysColumn
(column) & chr(13) & chr(10)
    col = myGrid.GetColumnColor (column)
    colstr = "<nicht gesetzt>"
    If (col >= 0) Then
        blue = CInt(col\65536)
        green = CInt((col-(blue*65536))\256)
        red = col-(blue*65536) - (green*256)
        colstr = col & "= rot: "&red&", grün: "&green&", blau: "&blue
    End If
    msg = msg & "aktuelle Spaltenfarbe: " & colstr & chr(13) & chr(10)
    col = myGrid.GetLineColor(row)
    colstr = "<nicht gesetzt>"
    If (col >= 0) Then
        blue = col\65536
        green = (col-(blue*65536))\256
        red = col-(blue*65536) - (green*256)
        colstr = col & "= rot: "&red&", grün: "&green&", blau: "&blue
    End If
    msg = msg & "aktuelle Zeilenfarbe: " & colstr & chr(13) & chr(10)
    msg = msg & "ist aktuelle Spalte read/only: " &
myGrid.GetColumnReadOnly (column) & chr(13) & chr(10)
    msg = msg & "aktuelle Spaltenbreite: " & myGrid.GetColumnWidth
(column) & chr(13) & chr(10)
    msg = msg & "Redraw aktiv: " & myGrid.isRedraw & chr(13) & chr(10)
    general.MsgBox msg
End If

End Sub

```

6.13. CWLReport

CWLReport
Eigenschaften
BSTR Name
short Type
BSTR HeaderFLags
BSTR MiddleFlags
short MultilinesLeft
BSTR Title
BSTR Description
BOOL ShowAbortWin
long Id

BOOL EnableDrilldown

Methoden

BOOL **Header** (VARIANT Flags)

short **Middle** (VARIANT Flags)

BOOL **Footer** (VARIANT Flags)

void **SetHiddenText** (short Type, BSTR Text, VARIANT where)

Events

OnPrintDrilldownItem (int ReportId, CWLEventResult DrillDownText, short View, short Var, BSTR ItemText)

OnCancel (int ReportId, CWLEventResult MayClose)

OnDrillDown (int ReportId, BSTR DrilldownText, BSTR Text)

6.13.1. Eigenschaften

Name [BSTR]

Der Name des Reports.

Type [short]

Wohin erfolgt die Ausgabe des Reports:

1... auf den Bildschirm

2... auf den Drucker

4... auf den Spooler

HeaderFlags [BSTR]

Die "Flags", die für den Kopf und den Fuß aktuell aktiv sind.

MiddleFlags [BSTR]

Die "Flags", die für den Mittelteil aktuell aktiv sind.

Title [BSTR]

Der Titel der Auswertung (max 50 Zeichen), die in der Spooldatei gespeichert wird, und z.B. im Fenster "Despool" in der Tabelle der gedruckten Dokumente angezeigt wird.

MultilinesLeft [short]

Die Anzahl der Zeilen eines Multilinetexts, die auf die nächste Seite umgebrochen werden.

Description [BSTR]

Die Beschreibung der Auswertung (max 100 Zeichen), die in der Spooldatei gespeichert wird, und z.B. im Fenster "Despool" in der Tabelle der gedruckten Dokumente angezeigt wird.

ShowAbortWin [BOOL]

Legt fest, ob während des Ausdrucks ein kleines Fenster mit dem Druckfortschritt angezeigt wird, mit dem der Druck abgebrochen werden kann.

–**Abbildung:** Anzeige des "AbortWin" beim Drucken

Id [long, read only]

Eine eindeutige Nummer, die während das Programm läuft für jede Auswertung eindeutig ist.

EnableDrillDown [BOOL]

Wird die Option auf TRUE gesetzt, dann können Drilldown-Einträge mit der Maus angeklickt werden..

6.13.2. Methoden

BOOL Header (VARIANT Flags)

Es wird der Kopfteil der Reportbeschreibung ausgegeben. Wird der Parameter *Flags* übergeben, dann werden diese Flags für die Ausgabe verwendet, im *CwlReport* gespeichert und können mit der Property *HeaderFlags* ausgelesen werden.

Parameter

Flags	Die zu verwendenden Flags (optional)
-------	--------------------------------------

Rückgabewert (BOOL)

Gibt False zurück, falls die Ausgabe durch einen Fehler abgebrochen wurde.

short Middle (VARIANT Flags)

Es wird der Mittelteil der Reportbeschreibung ausgegeben. Wird der Parameter *Flags* übergeben, dann werden diese Flags für die Ausgabe verwendet, im *CwlReport* gespeichert und können mit der Property *MiddleFlags* ausgelesen werden.

Parameter

Flags	Die zu verwendenden Flags (optional)
-------	--------------------------------------

Rückgabewert (short)

0... Ausgabe wurde durchgeführt

1... ein Fehler ist aufgetreten

2... die Ausgabe muss auf der nächsten Seite wiederholt werden, weil sie auf der aktuellen Seite nicht mehr Platz fand.

Wurde ein Multilinetext ausgegeben, könnte es sein, dass die Werte des Mittelteils zwar gedruckt wurden, der Text aber auf die nächste Seite umgebrochen wurde. Mit der Property *MuliLinesLeft* kann dies überprüft werden. In diesem Fall können für den ersten Mittelteil auf der nächsten Seite bereits die nächsten Variablen gesetzt werden, die umgebrochenen Multiline-Zeilen werden automatisch zu Beginn der neuen Seite ausgegeben.

BOOL Footer (VARIANT Flags)

Es wird der Fußteil der Reportbeschreibung ausgegeben. Wird der Parameter *Flags* übergeben, dann werden diese Flags für die Ausgabe verwendet, im *CwlReport* gespeichert und können mit der Property *HeaderFlags* ausgelesen werden.

Parameter

Flags	Die zu verwendenden Flags (optional)
-------	--------------------------------------

Rückgabewert (BOOL)

Gibt False zurück, falls die Ausgabe durch einen Fehler abgebrochen wurde.

void setHiddenText (short Type, BSTR Text, VARIANT where)

Für die Ausgabe von Kopf, Mittelteil und Fuß kann jeweils ein bestimmter Text für den Hiddentext für Drilldown-Elemente gesetzt werden. Der "Hiddentext" ist der versteckte Text, der das Ziel des Drilldown-Elements darstellt (im Gegensatz zum angezeigten Text). Dieser Text wird dann in jedem Drilldown-Element des Ausgabebereichs verwendet.

Parameter

type	Der Typ des "Hiddentext". Der Typ sollte
------	--

	immer 0 sein.
Text	Der Text, der im "Hiddentext" gespeichert wird
where	Wo wird der "Hiddentext" gesetzt 0... Kopf 1... Mittelteil 2... Fuß

6.13.3. Events

OnPrintDrilldownItem (int ReportId, CWLEventResult DrillDownText, short View, short Var, BSTR ItemText)

Feuert bei der Ausgabe eines Eintrags, der als Drilldown-Eintrag gekennzeichnet ist. Damit kann jedes Drilldownelement einen eigenen "HiddenText" erhalten.

Parameter

ReportId	Die Id des Reports, für den das Event gesendet wird (jeder Report hat eine eindeutige Id)
DrillDownText	Der Text, der im Drilldownelement aktuell gesetzt ist, und der hier verändert werden kann.
View	die Tabelle, die den ausgegebenen Wert beinhaltet
Var	die Variable innerhalb der Tabelle, die mit dem Eintrag ausgegeben wird
ItemText	der Text des ausgegebenen Eintrags

OnCancel (int ReportId, CWLEventResult MayClose)

Dieses Event feuert, wenn auf den Stop-Button in der Auswertung gedrückt wird, oder das Auswertungsfenster über den Schließen-Button (rotes X) geschlossen wird. Wird eine Ausgabe am Drucker ausgegeben und es ist das AbortWin eingeschaltet (Eigenschaft *ShowAbortWin*), dann wird beim Klick auf den Abbruch-Button ebenfalls das Event ausgelöst.

Der Stop-Button kann auch gedrückt werden, während die Ausgabe noch arbeitet. Wenn das Programm das Event unterstützt, kann es damit eine Ausgabe auch während des Druckes abbrechen (es muss eine globale Variable definiert sein, die in der Schleife, die die Auswertung ausgibt, abgeprüft wird und im Event gesetzt wird).

Parameter

ReportId	Die Id des Reports, für den das Event gesendet wird (jeder Report hat eine eindeutige Id)
MayClose	Wird der Wert auf TRUE gesetzt (MayClose.Value = true), dann wird die Auswertung geschlossen.

OnDrillDown (int ReportId, BSTR DrilldownText, BSTR Text)

Dieses Event wird gesendet, wenn der Anwender auf einen Drilldown-Link klickt. Voraussetzung ist, dass die Eigenschaft *EnableDrillDown* auf TRUE gesetzt ist.

Parameter

ReportId	Die Id des Reports, für den das Event gesendet wird (jeder Report hat eine eindeutige Id)
DrillDownText	Der Text, der im Drilldownelement aktuell gesetzt ist.
Text	der sichtbare Text des Eintrags

7. Konstanten

7.1. CWLApplicationNr

ID für die Applikationen, die aktiv sein können.

Name	Wert
cwlMAIN	0
cwlFIBU	1
cwlFAKT	2
cwlLOHN A	3
cwlLIST	4
cwlKORE	5
cwlANBU	6
cwlINFO	11
CwlLOHN D	18
cwlPROD	20

7.2. CWLWindowTypes

Typ eines Fensters. Siehe Eigenschaft **Type** von Objekten der Klasse **CWLWindow**.

Name	Wert	Beschreibung
winStandardType	0	Standard CWL Fenster
winPreviewType	1	Preview Fenster
WinScriptType	2	UserForm eines Scripts

7.3. CWLControlTypes

Typ eines Elements (Controls) in einem Fenster. Siehe Eigenschaft **Type** von Objekten der Klasse **CWLfgControl**.

Name	Wert	VarType	Beschreibung
cwlControlEditString	1	8	
cwlControlEditInteger	2	3	
cwlControlEditFloat	3	5	
cwlControlEditDouble	4	5	
cwlControlEditUppercase	5	8	
cwlControlEditDate	6	7	
cwlControlEditMultiline	7	8	
cwlControlEditPassword	8	-	kein Content
cwlControlEditTimespan	9	3	
cwlControlButton	11	-	kein Content
cwlControlCheckbox	12	8	"1"=on "0"=off
cwlControlRadioButton	13	8	"1" für das Element der Gruppe, das ausgewählt ist (die anderen "0")
cwlControlListbox	15	-	

cwlControlTree	18	-	
cwlControlStaticString	21	-	kein Content
cwlControlStaticInteger	22	-	kein Content
cwlControlStaticFloat	23	-	kein Content
cwlControlStaticDouble	24	-	kein Content
cwlControlStaticUppercase	25	-	kein Content
cwlControlStaticDate	26	-	kein Content
cwlControlStaticTimespan	29	-	kein Content
cwlControlFrame	30	-	kein Content
cwlControlCombobox	31	-	kein Content
cwlControlGrid	35	-	kein Content
cwlControlPreview	36	-	kein Content
cwlControlSpreadsheet	37	-	Kein Content

7.4. CWLSpoolItemType

Typ eines Elements in einer Spoolpreview. Siehe Eigenschaft Type von Objekten der Klasse CWLPreviewPageItem.

Name	Wert	Beschreibung
cwlSpoolItemText	84	konstanter Text wie er im Formular hinterlegt war
cwlSpoolItemVar	86	variabler Text wird beim Ausdruck mit der entsprechende Variable befüllt
cwlSpoolItemLookup	85	'Lookup' Element, das den in der Variable hinterlegten Text in einer anderen Tabelle sucht und den dort gefundenen Wert anzeigt
cwlSpoolItemGraphic	71	Ein Grafik Element (Bitmap)
cwlSpoolItemObject	79	Diagramm Element (Balken, Torten oder Liniendiagramm)
cwlSpoolItemBar	66	Eine grafische Prozentanzeige als horizontaler Balken
cwlSpoolItemControl	83	Das Element enthält nicht druckbare Anweisungen, die beim Ausdruck ausgeführt werden (z.B. Druckerwechsel)
cwlSpoolItemFormula	70	Eine Formel, die beim Ausdruck ausgeführt wird
cwlSpoolItemLine	76	Eine horizontale oder vertikale Linie
cwlSpoolItemRect	82	Ein Rechteck (gefüllt oder nur umrahmt)
cwlSpoolItemMultiline	77	Ein Text, der in der definierten Breite verschieden hoch angezeigt wird (Umbruch)

7.5. CWLSpoolPreviewItemFlag

Typ des hidden Text Elements in einem CwlPreviewPageItem.

Name	Wert	Beschreibung
cwlHiddenflagDrilldown	0	DrillDown Items können einen Text hinterlegt haben
cwlHiddenflagGroup	1	GroupItems haben normalerweise kein Text hinterlegt
cwlHiddenflagEdit	2	Items die editiert werden (z.B. im Quick Erfassen) haben normalerweise kein Text hinterlegt
cwlHiddenflagUser	3	können einen Text hinterlegt haben

7.6. CWLAlignements

Typ eines Elements in einer Spoolpreview. Siehe Eigenschaft Alignment von Objekten der Klasse CwlPreviewPageItem.

Name	Wert	Beschreibung
cwlAlignLeft	108	linksbündig
cwlAlignRight	114	rechtsbündig
cwlAlignCenter	122	zentriert

7.7. CWLScriptWindowType

Typ eines Systemskripts. Siehe mode Parameter der Methode CWLStart.RunFormScript.

Name	Wert	Beschreibung
cwlScriptWindowStandard	0	wie alle normalen Programmfenster (auch CTK Fenster solange sie nicht modal gestartet werden), wird beim Modulwechsel versteckt
cwlScriptWindowModal	1	modales Fenster, erst nach dem Schließen des Fensters, kann in der Applikation weitergearbeitet werden
cwlScriptWindowSystem	2	Script Fenster, das über allen anderen Fenstern schwebt und beim Modulwechsel immer sichtbar bleibt. Fenster dieses Typs haben keine interne ID und können im MayCloseWindow - Event nicht identifiziert werden.

7.8. CWLSystemServerType

Typ einer Systemdatenbank. Siehe *what* Parameter der Methode CWLCompany.GetSystemConnection.

Name	Wert	Beschreibung
cwlSystemServerSRV	2	Systemdatenbank für die Datenbankverbindungen, Benutzer, Benutzerberechtigungen, MSM, Audit, Benutzergruppen, usw.
cwlSystemServerARC	4	Systemdatenbank für die Archivierungstabellen
cwlSystemServerPDB	8	Systemdatenbank für die Formulare und Fensterbeschreibungen
cwlSystemServerCMP	16	Systemdatenbank für die mandantenunabhängigen Tabellen
cwlSystemServerLOHN	32	Systemdatenbank für den österreichischen Lohn
cwlSystemServerLOHD	64	Systemdatenbank für den deutschen Lohn
cwlSystemServerPOWERREPORT	128	Systemdatenbank für Datenquellen
cwlSystemServerAUDIT	256	Systemdatenbank für das ausgelagerte Variablenaudit
cwlSystemArchiveData	512	Die Datenbank für die Archivedaten. Im Normalfall in der Systemdatenbank, könnten aber auch ausgelagert sein.

7.9. CWLDbConnectionType

Typ einer Datenbankverbindung.

Name	Wert	Beschreibung
cwlDbConnectionTypeDAO	0	Microsoft Access Datenbankformat
cwlDbConnectionTypeSQL	1	Microsoft SQL Server Datenbankformat
cwlDbConnectionTypePOS	4	PostgreSQL Datenbankformat

7.10. CWLGridColumnFlags

Attribut für neue Spalten einer Bildschirmtable.

Name	Wert	Beschreibung
SORTFLAG	1	Spalte ist sortierbar
HIDEFLAG	4	Spalte kann versteckt werden
READONLYFLAG	8	Spalte ist read-only
MOVEFLAG	16	Spalte kann verschoben werden
SIZEFLAG	32	Spaltengröße kann verändert

		werden
INVISIBLEFLAG	64	Spalte ist unsichtbar und kann nicht angezeigt werden
COMPANYYEARFLAG	256	Spalte enthält ein Wirtschaftsjahr, welches automatisch bei unterschiedlichen Kalendern umgerechnet wird

8. Tipps und Tricks

8.1. Bearbeiten von Scripts bei Kundeninstallationen

Wenn beim Kunden ein bestehendes MDP-Projekt eingerichtet ist (es ist nur eine MDP-Runtime-Lizenz vorhanden), und dort Änderungen vorgenommen werden müssen, dann kann dies durch drei Varianten erreicht werden:

➤ **Einspielen MDP-Developer - Lizenz**

Wenn die entsprechende Developer-Lizenz eingespielt wird, kann das Script normal verändert werden.

➤ **Script wird mit STRG+SHIFT+Editieren aufgerufen**

Wenn beim Anklicken des Editieren-Buttons die Tastenkombination STRG+SHIFT gedrückt wird, dann kann das Script auch mit einer MDP-Runtime-Lizenz geöffnet werden.

➤ **mesonic.ini - Eintrag**

Mit dem Eintrag

```
[MDPLicense]
AllowEditForRuntimeOnly=1
```

kann auch mit der MDP-Runtime-Lizenz ein Script durch Anklicken des Editieren-Buttons geöffnet werden.

8.2. Kann von extern ein WinLine-Fenster angesprochen werden?

Wenn das CWLSTART läuft (und nur dann) und eine MDP Lizenz vorhanden ist (Runtime genügt), kann das CWL Objekt verwendet werden, und damit kann auch von extern alles gemacht werden, das normalerweise nur innerhalb der MDP Scripte verwendet wird (z.B. auch Makros starten).

Beispiel für VB-Script:

```
dim appl
set appl = createobject ("cwlstart.application" )

if err then
    msgbox "Das CWLstart.exe ist nicht gestartet. Fehler: " & Err.Description
end if

' Dieser Befehl funktioniert auch ohne MDP Lizenz
msgbox "Der Applikationsname ist: " & appl.Name

' auf die FIBU umschalten (dafür ist schon eine MDP Lizenz notwendig)
appl.ActivateModule 1
```

8.3. Makros aus einem Script heraus aufrufen

Im nachfolgendem Beispiel wird dargestellt, wie man aus einem Script heraus ein Makro starten kann, wobei auch Parameter mit übergeben werden sollen.

Beispiel:

In einem Systemsript soll abhängig von den Eingaben im Script ein Makro aufgerufen werden, das die Eingaben als Parameter übergeben bekommen soll.

```
Sub CommandButton1_Click()
    Dim params(1)      ' Platz für zwei Parameter (Index 0 und 1)
    ' die Feldwerte setzen (hier im Beispiel mit Teststrings)
    params(0) = "der erste Parameter"
    params(1) = "der zweite Parameter"

    pParams = params  ' diese Zuweisung verwandelt das Feld params in
den VARIANT          ' pParams, der an das Makro übergeben werden kann

    'Aufruf des Makros
    MacroCommands.MRunMacro "AUSWERTUNG", pParams
End Sub
```

Im Makro müssen die Parameter wieder extrahiert werden:

```
Sub RunMacro
    ' durch die Zuweisung wird aus dem VARIANT
    ' wieder ein richtiges Array
    inparams = MParameters

    ' die Werte vorbelegen, falls keine Parameter übergeben wurden
    value1 = "erster"
    value2 = "zweiter"

    ' Überprüfen ob 2 Parameter übergeben wurden

    ' per default sind in den Makroparametern von 0 bis 19
    ' die view0 - Parameter gespeichert, ab 20 beginnen die
    ' Parameter, die extra übergeben wurden
    If ubound(inparams) >= 21 Then      ' mindestens 2 Parameter ?
        value1 = inparams(20)
        value2 = inparams(21)
    End If
    ...
    ...
End Sub
```

8.4. Optionale Steuerung der rechten Maustaste

Anforderung:

Export der Tabellen bzw. andere Option auf der rechten Maustaste im Matchcode soll gesperrt werden - Benutzer- bzw. Benutzergruppenspezifisch. Generell Optionen über die rechte Maustaste freigeben oder nicht.

Lösungsansatz:

Über MDP-Programmierung können Einstellungen für das Verhalten der "rechten Maustaste" vorgenommen (programmiert) werden. Damit können dann bestimmte Funktionen der rechten Maustaste (z.B. Tabelle Exportieren, In die Zwischenablage kopieren etc.) für bestimmte Benutzergruppen ausgenommen werden, d.h. diese Funktionen werden dann nur mehr grayed dargestellt und haben somit keine Funktion mehr.

Umsetzung:**➤ OnContextMenu - Event**

Es gibt im Anwendungsobjekt das neue Event OnContextMenu, das bei der Anzeige eines Kontextmenüs für jeden Menüpunkt aufgerufen wird. Bei jedem Menüpunkt kann die Anzeige grayt werden, indem bResult.value auf false gesetzt wird.

Beispiel:

```
Sub CWLStart_OnContextMenu(AppNr, WindowId, FgId, MenuText, MenuId, bResult)

    If Cwlstart.Currentuser.Group = 1 Then
        If MenuId = 14291 Or MenuId = 14709 Or MenuId = 13294 Then
            Bresult.Value = FALSE
        End If
    End If

End Sub
```

Parameter der Funktion:**➤ AppNr:**

die aktuelle Anwendung (z.B. 1... FIBU)

➤ WindowId:

das aktuelle Fenster (0, wenn das Kontextmenü nicht im Fenster aufgerufen wurde)

➤ FgId:

das aktuelle Fensterelement (0, wenn das Kontextmenü nicht im Fenster aufgerufen wurde)

➤ MenuText:

der angezeigte Name des Menüpunkts

➤ MenuId:

die Id des Menüpunkts z.B. 14325 für Spalten anzeigen/verstecken oder 14709 für Tabelle Exportieren.

➤ bResult:

mit bResult.value = false kann der Menüpunkt grayt werden

Nachfolgend ein Beispiel, mit dem man zuerst alle Funktionen einer rechten Maustaste auslesen kann und dann z.B. die Einträge für Spalten anzeigen/verstecken und Tabelle Exportieren für die Benutzergruppe 1 "deaktivieren" kann:

```
Sub Cwlstart_oncontextmenu(Appnr, Windowid, Fgid, Menutext, Menuid, Bresult)

    If Cwlstart.Currentuser.Group = 1 Then 'Wenn der Benutzer in der
Benutzergruppe 1 ist
        MsgBox Menuid & " " & Menutext 'wird jeder Eintrag in der rechten Maustaste
mit Nummer und Bezeichnung ausgegeben

        If Menuid = 14325 Or Menuid = 14709 Then
            Bresult.Value = False
        End If

    End If

End Sub
```

➤ Objekt CWLUser

Das Objekt CWLUser wurde erstellt, damit aus dem Anwendungsobjekt mit CWLStart.CurrentUser zugegriffen werden kann (vergleiche auch die Anwendung im obigen Beispiel).

Das Objekt unterstützt die folgenden read only Properties:

- ☐ Name
- ☐ Number
- ☐ OrigNumber (Nummer des original Benutzers z.B. bei einem Schattenbenutzer)
- ☐ Priority
- ☐ Group
- ☐ Demo
- ☐ Company
- ☐ Type
- ☐ CWLUserNo
- ☐ Account
- ☐ Employee
- ☐ Salesman
- ☐ WEBCompany
- ☐ UserLocked
- ☐ Language
- ☐ WTRRecord
- ☐ Customer
- ☐ GUID
- ☐ GUID2
- ☐ PasswordExpiresOn
- ☐ PasswordExpiresInDays
- ☐ LastActivity
- ☐ LongName
- ☐ SMTPAdress
- ☐ Registered

Beispiel:

Das folgende Script für zur folgenden Ausgabe:

```
Msg = "Current User:" & Chr(13) & Chr(10)
Msg = Msg & "Name: " & Cwlstart.Currentuser.Name & Chr(13) & Chr(10)
Msg = Msg & "Konto: " & Cwlstart.Currentuser.Account & Chr(13) & Chr(10)
Msg = Msg & "Mandant: " & Cwlstart.Currentuser.Company & Chr(13) & Chr(10)
Msg = Msg & "Laufkunde: " & Cwlstart.Currentuser.Customer & Chr(13) & Chr(10)
Msg = Msg & "CWL User Nummer: " & Cwlstart.Currentuser.Cwluserno & Chr(13) & Chr(10)
Msg = Msg & "Demo: " & Cwlstart.Currentuser.Demo & Chr(13) & Chr(10)
Msg = Msg & "AN: " & Cwlstart.Currentuser.Employee & Chr(13) & Chr(10)
Msg = Msg & "Gruppe: " & Cwlstart.Currentuser.Group & Chr(13) & Chr(10)
Msg = Msg & "Guid: " & Cwlstart.Currentuser.Guid & Chr(13) & Chr(10)
Msg = Msg & "Guid2: " & Cwlstart.Currentuser.Guid2 & Chr(13) & Chr(10)
Msg = Msg & "Sprache: " & Cwlstart.Currentuser.Language & Chr(13) & Chr(10)
Msg = Msg & "Letzte Aktiv.: " & Cwlstart.Currentuser.Lastactivity & Chr(13) & Chr(10)
Msg = Msg & "Lange Name: " & Cwlstart.Currentuser.Longname & Chr(13) & Chr(10)
Msg = Msg & "Nummer: " & Cwlstart.Currentuser.Number & Chr(13) & Chr(10)
Msg = Msg & "Password läuft ab in Tagen: " & Cwlstart.Currentuser.Passwordexpiresindays & Chr(13) & Chr(10)
Msg = Msg & "Passwort läuft ab am: " & Cwlstart.Currentuser.Passwordexpireson & Chr(13) & Chr(10)
Msg = Msg & "Priorität: " & Cwlstart.Currentuser.Priority & Chr(13) & Chr(10)
Msg = Msg & "Registered: " & Cwlstart.Currentuser.Registered & Chr(13) & Chr(10)
```

General.Msgbox Msg

